**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**

**TRƯỜNG ĐẠI HỌC BÁCH KHOA**

**KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**

# BÁO CÁO BÀI TẬP LỚN

# PHÁT TRIỂN ỨNG DỤNG IOT

# Chủ đề:

| | | |
|---|---|---|
| **Giảng viên hướng dẫn** | **Lê Trọng Nhân** | |
| **Sinh viên thực hiện 1** | **Đoàn Trần Cao Trí** | **2470510** |
| **Sinh viên thực hiện 2** | **Đặng Nguyễn Tấn Tài** | **2570357** |

**Lớp: CH01**

**Thành phố Hồ Chí Minh, 04/2026**

# Mục lục


## I Bối cảnh, mục tiêu và phạm vi

**1 Giới thiệu đề tài và bối cảnh . . . . . 2**

*1.1 Bài toán đặt ra . . . . . 2*

*1.2 Mục tiêu của đồ án . . . . . 2*

*1.3 Phạm vi và giới hạn . . . . . 2*

*1.4 Phương pháp trình bày báo cáo . . . . . 3*

*1.5 Nguồn mã và khả năng đối chiếu . . . . . 3*

## II Yêu cầu và định hướng thiết kế

**2 Yêu cầu hệ thống và định hướng thiết kế . . . . . 6**

*2.1 Định hướng từ đề bài môn học . . . . . 6*

*2.2 Yêu cầu chức năng . . . . . 6*

*2.3 Yêu cầu phi chức năng . . . . . 6*

*2.4 Lý do chọn kiến trúc nhiều sender – một receiver . . . . . 7*

## III Kiến trúc và thiết kế các khối

**3 Kiến trúc tổng thể và phân rã hệ thống . . . . . 9**

*3.1 Bức tranh tổng quát của hệ thống. . . . . 9*

*3.2 Mô hình data plane và control plane . . . . . 9*

*3.3 Lợi ích của việc phân rã module . . . . . 10*

*3.4 Bảng so sánh phần cứng và phần mềm của hai khối . . . . . 11*

**4 Thiết kế chi tiết khối Sender . . . . . 12**

*4.1 Vai trò của sender trong toàn hệ thống . . . . . 12*

*4.2 Luồng khởi tạo sender . . . . . 12*

*4.3 Mô phỏng dữ liệu sensor . . . . . 13*

*4.4 TinyML inference ở edge . . . . . 13*

*4.5 Lớp mạng và sender transport* . . . . . 14
*4.6 Giao diện web cục bộ của sender* . . . . . 15
**5 Thiết kế chi tiết khối Receiver** . . . . . 16
*5.1 Receiver như một gateway thực thụ* . . . . . 16
*5.2 Luồng khởi tạo receiver* . . . . . 17
*5.3 Mô hình dữ liệu trung tâm của receiver* . . . . . 17
*5.4 Known sender và unknown device* . . . . . 18
*5.5 Cơ chế online/offline* . . . . . 19
*5.6 Web UI quản trị của receiver* . . . . . 20

**IV Giao thức, luồng điều khiển và tích hợp cloud**

**6 Thiết kế giao thức truyền thông** . . . . . 22
*6.1 Lý do chọn TCP nội bộ* . . . . . 22
*6.2 Định dạng packet TCP* . . . . . 22
*6.3 Chi tiết packet telemetry* . . . . . 23
*6.4 Chi tiết packet config và ACK* . . . . . 23
*6.5 Giao thức MQTT với CoreIoT* . . . . . 23
**7 Thiết kế luồng điều khiển cấu hình và RPC** . . . . . 24
*7.1 Tại sao cần đồng bộ cấu hình hai chiều* . . . . . 24
*7.2 Luồng đổi threshold từ receiver UI* . . . . . 25
*7.3 Luồng đổi threshold từ RPC cloud* . . . . . 25
*7.4 Retry và timeout* . . . . . 26
**8 Tích hợp CoreIoT và vai trò của cloud** . . . . . 27
*8.1 Receiver như một MQTT gateway* . . . . . 27
*8.2 Giá trị của tầng cloud đối với hệ thống* . . . . . 27
*8.3 Ý nghĩa của việc hỗ trợ RPC* . . . . . 28

**V Ràng buộc, độ tin cậy và phân tích mã nguồn**

**9 Ràng buộc kỹ thuật và quyết định thiết kế** . . . . . 30
*9.1 Bảng hằng số hệ thống và ý nghĩa vận hành* . . . . . 30
*9.2 Trade-off của các quyết định kiến trúc* . . . . . 30

**10 Tình huống lỗi, cơ chế phòng vệ và bằng chứng kỹ thuật . . . . . 31**

*10.1 Các tình huống lỗi điển hình . . . . . 31*

*10.2 Cơ chế phòng vệ trong mã nguồn . . . . . 31*

*10.3 Bảng đối chiếu luận điểm và bằng chứng mã nguồn . . . . . 31*

*10.4 Ghi chú hình cần chèn ở chương này . . . . . 32*

**11 Phân tích mã nguồn theo module và theo dòng ý nghĩa . . . . . 33**

*11.1 Tổ chức phân tích mã nguồn . . . . . 33*

*11.2 Trích đoạn 1: vòng lặp chính của sender. . . . . 33*

*11.3 Trích đoạn 2: suy luận TinyML trên sender . . . . . 34*

*11.4 Trích đoạn 3: gửi telemetry qua TCP . . . . . 35*

*11.5 Trích đoạn 4: xử lý telemetry ở receiver . . . . . 35*

*11.6 Trích đoạn 5: gửi cấu hình từ receiver xuống sender. . . . . 36*

*11.7 Trích đoạn 6: publish RPC response lên CoreIoT. . . . . 37*

## VI Giao diện, đánh giá và kết luận

**12 Thiết kế web UI và trải nghiệm vận hành . . . . . 39**

*12.1 Web UI sender . . . . . 39*

*12.2 Web UI receiver . . . . . 39*

*12.3 Ảnh thực nghiệm giao diện và log. . . . . 39*

**13 Kịch bản thực nghiệm và kế hoạch đánh giá . . . . . 40**

*13.1 Mục tiêu đánh giá. . . . . 40*

*13.2 Danh sách test case đề xuất . . . . . 41*

*13.3 Chỉ số quan sát cần ghi lại. . . . . 42*

*13.4 Đánh giá ưu điểm của hệ thống. . . . . 43*

*13.5 Hạn chế hiện tại. . . . . 44*

**14 Danh sách ghi chú hình minh họa cần chèn . . . . . 44**

*14.1 Nhóm hình kiến trúc và mạng. . . . . 44*

**15 Bàn luận kỹ thuật và hướng phát triển. . . . . 45**

*15.1 Vì sao kiến trúc này phù hợp cho môn học . . . . . 45*

*15.2 Khả năng mở rộng trong tương lai . . . . . 45*

*15.3 Khả năng áp dụng thực tế . . . . . 46*

**16 Kết luận . . . . . 46**

## VII Phụ lục

**1 Phụ lục A: Trích đoạn mã nguồn tiêu biểu và diễn giải ngắn . . . . . 48**

**2 Phụ lục B: Hình ảnh thực nghiệm bổ sung. . . . . 52**

# Danh sách hình vẽ


1 Kiến trúc tổng thể của hệ thống IoT Application theo mô hình nhiều sender, một receiver gateway và tầng cloud CoreIoT (demo hiện tại dùng hai sender). . . . 4

2 Sơ đồ mạng nội bộ cho thấy receiver vừa phát SoftAP vừa kết nối Wi-Fi STA, trong khi sender suy ra địa chỉ receiver ở host .100. . . . . 7

3 Sơ đồ tách *data plane* và *control plane* giúp nhận diện rõ hai loại luồng trong hệ thống. . . . . 9

4 Kiến trúc module của sender gồm các khối trạng thái, mô phỏng, AI, transport, lưu trữ và giao diện web. . . . . 10

5 Kiến trúc module của receiver gồm runtime, transport, cloud gateway, lưu trữ và giao diện web. . . . . 11

6 Flowchart phần khởi tạo và vòng lặp sender. Sơ đồ cho thấy sender liên tục mô phỏng cảm biến, chạy AI và duy trì mạng. . . . . 13

7 Pipeline TinyML trên sender: từ dữ liệu mô phỏng sang tensor input, qua bộ thông dịch TFLite Micro, rồi trả ra điểm bất thường và nhãn trạng thái. . . . . 14

8 Mô hình logic của TFLite được sử dụng ở mức báo cáo: hai đầu vào đặc trưng và một đầu ra điểm bất thường. . . . . 14

9 Cơ chế WebSocket snapshot của sender và receiver, cho phép web UI nhận trạng thái thời gian thực mà không cần tải lại trang. . . . . 15

10 Ảnh thực nghiệm giao diện web của sender A. . . . . 16

11 Ảnh thực nghiệm giao diện web của sender B. . . . . 16

12 Flowchart phần khởi tạo và vòng lặp receiver. Sơ đồ cho thấy receiver đồng thời xử lý DNS, TCP, Wi-Fi, MQTT và snapshot cho UI. . . . . 17

13 Mô hình dữ liệu trung tâm của receiver gồm cấu hình gateway, trạng thái sender đã đăng ký, thiết bị lạ và các kết nối TCP đang sống. . . . . 18

14 Flowchart phân loại telemetry theo mã thiết bị: sender đã đăng ký được cập nhật vào bảng trạng thái, còn sender chưa đăng ký được đưa vào danh sách thiết bị lạ. 19

15 Flowchart chuyển trạng thái online/offline của sender trên receiver dựa vào timeout. . . . . 20

16 Ảnh thực nghiệm giao diện quản trị của receiver. . . . . 20

17 Bảng tổng hợp ba loại message TCP và các trường dữ liệu chính được trao đổi giữa sender và receiver. . . . . 22
18 Sequence diagram mô tả nhiều sender gửi telemetry lên receiver, receiver cập nhật trạng thái và tiếp tục publish lên cloud. . . . . 23
19 Bảng tổng hợp các topic MQTT gateway mà receiver dùng để làm việc với CoreIoT. . . . . 24
20 Sequence diagram mô tả receiver đại diện sender khi gửi connect, telemetry, attribute và xử lý RPC qua MQTT gateway. . . . . 24
21 Sequence diagram mô tả quá trình receiver gửi packet cấu hình xuống sender và nhận lại gói xác nhận. Cùng cơ chế này cũng được dùng khi có RPC từ cloud. 25
22 State machine đồng bộ cấu hình giữa receiver và sender. . . . . 26
23 Flowchart cơ chế retry cấu hình của receiver khi sender chưa ACK trong thời gian cho phép. . . . . 27
24 Ảnh thực nghiệm dashboard CoreIoT sau khi receiver publish dữ liệu. . . . . 28
25 Sơ đồ state machine đồng bộ cấu hình: pending, awaiting ACK, timeout, retry, success/fail. . . . . 32
26 Sequence xử lý lỗi RPC khi sender offline và phản hồi lại cloud. . . . . 32
27 Flowchart phân bổ slot TCP nhiều sender và cơ chế chống starvation. . . . . 33
28 Ảnh bảng trạng thái sender trên receiver. . . . . 39
29 Ảnh log serial của sender A. . . . . 40
30 Ảnh log serial của receiver. . . . . 40
31 Ảnh minh họa test case đổi threshold. . . . . 43
32 Ảnh minh họa test case sender offline rồi reconnect. . . . . 43
33 Cấu trúc lưu trữ LittleFS của sender và receiver. Hình này giúp người đọc hiểu rõ những tệp nào là cấu hình và những tệp nào dùng để phục vụ web UI. . . . . 44
34 Bảng topic MQTT dùng bởi receiver khi giao tiếp với CoreIoT. . . . . 45
35 Sequence end-to-end tổng hợp cả dữ liệu đi lên, điều khiển đi xuống và snapshot tới browser. . . . . 45
36 Bố trí phần cứng của hệ thống. . . . . 52
37 Sender ở trạng thái ANOMALY. . . . . 53
38 Xác nhận RPC thay đổi threshold thành công. . . . . 54

**Phần I**

# Bối cảnh, mục tiêu và phạm vi

# 1 Giới thiệu đề tài và bối cảnh

## *1.1 Bài toán đặt ra*

Đề tài được xây dựng trong bối cảnh các hệ thống IoT hiện đại ngày càng dịch chuyển từ mô hình *đọc cảm biến rồi gửi thẳng lên cloud* sang mô hình phân tầng gồm thiết bị biên, gateway cục bộ và nền tảng cloud. Với cách tổ chức này, dữ liệu không chỉ được truyền đi một chiều mà còn được xử lý cục bộ ở edge, được quản lý và đồng bộ tập trung ở gateway, sau đó mới được đưa lên nền tảng cloud để trực quan hóa và điều khiển từ xa. Cách tiếp cận này phản ánh tương đối đúng với nhiều hệ thống thực tế trong công nghiệp, nông nghiệp thông minh và smart building.

Trong phạm vi đồ án, nhóm triển khai kiến trúc nhiều sender và một receiver gateway; cấu hình demo hiện tại dùng hai sender để thuận tiện kiểm thử và trình diễn. Mỗi sender giả lập một điểm đo môi trường có khả năng phát sinh dữ liệu nhiệt độ, độ ẩm và vị trí GPS. Trên sender, dữ liệu cảm biến được đưa trực tiếp vào mô hình TinyML để tính ra điểm bất thường `ai_score` và nhãn `NORMAL`/`ANOMALY`. Receiver không phải là một thiết bị hiển thị đơn thuần mà là một gateway thực thụ: lắng nghe TCP server, quản lý trạng thái của nhiều sender, đồng bộ cấu hình hai chiều và đại diện cho các sender khi làm việc với CoreIoT.

## *1.2 Mục tiêu của đồ án*

Mục tiêu kỹ thuật của đồ án gồm năm nhóm chính. Thứ nhất, thiết kế được kiến trúc nhiều sender một receiver có phân vai rõ ràng và có thể mở rộng tăng số node trong tương lai. Thứ hai, đưa một mô hình TinyML chạy trực tiếp trên sender nhằm thực hiện suy luận cục bộ ở edge thay vì phụ thuộc hoàn toàn vào cloud. Thứ ba, tổ chức được luồng cấu hình hai chiều từ giao diện web hoặc RPC cloud xuống sender thông qua receiver. Thứ tư, xây dựng web UI cục bộ cho cả sender và receiver để người dùng có thể quan sát và điều khiển hệ thống không cần công cụ phụ trợ. Thứ năm, kết nối receiver với CoreIoT thông qua MQTT gateway để minh họa luồng dữ liệu đầu cuối từ edge đến cloud.

Mục tiêu học thuật của báo cáo là chuyển toàn bộ nội dung trên thành một tài liệu kỹ thuật có cấu trúc chặt chẽ, giàu hình minh họa, giải thích được quyết định thiết kế và thể hiện rõ cách mã nguồn hiện thực hóa các quyết định đó. Báo cáo vì vậy cần cân bằng giữa *kiến trúc hệ thống*, *luồng hoạt động* và *phân tích mã nguồn có chọn lọc*.

## *1.3 Phạm vi và giới hạn*

Phạm vi đồ án tập trung vào tầng firmware và gateway, bao gồm mô phỏng dữ liệu sensor, TinyML inference, web UI cục bộ, TCP nội bộ và tích hợp CoreIoT. Báo cáo không đi sâu vào phần huấn luyện mô hình TinyML ở phía máy tính ngoài, cũng không xây dựng dashboard web cloud tùy biến ngoài phần hiển thị và RPC sẵn có của CoreIoT. Dữ liệu môi trường và GPS

hiện được mô phỏng theo thời gian, nên mục tiêu chính của nhóm là chứng minh đúng luồng kỹ thuật và kiến trúc hệ thống, thay vì tối ưu độ chính xác của cảm biến thật.

## *1.4 Phương pháp trình bày báo cáo*

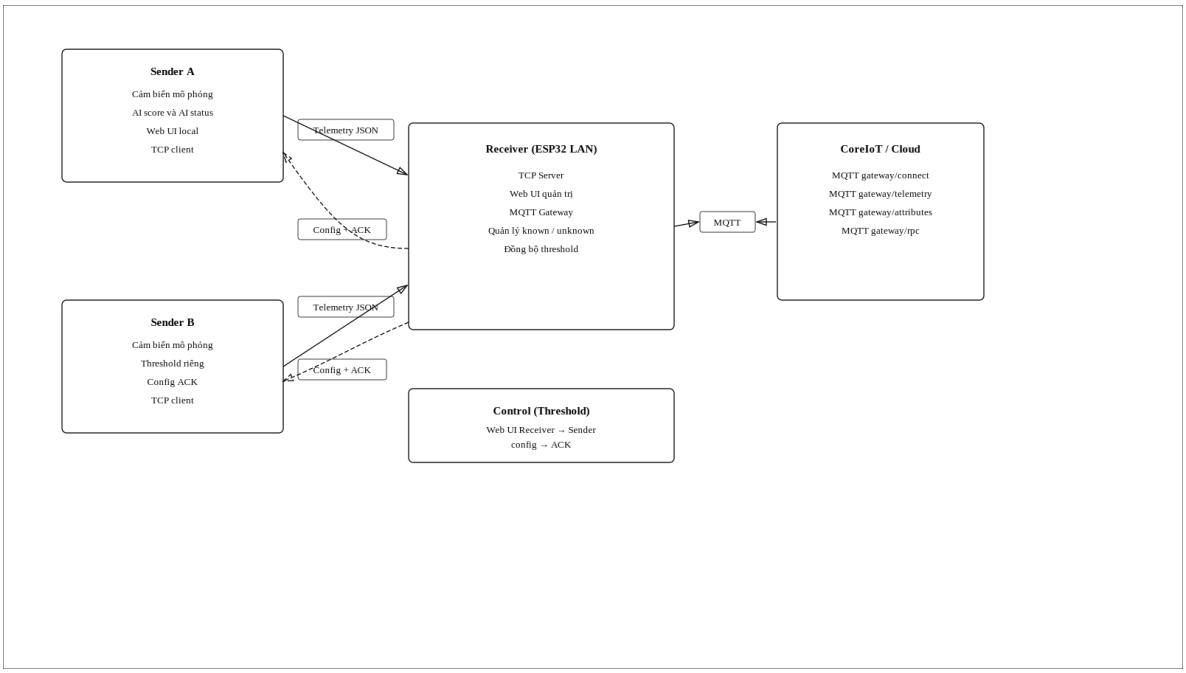
Theo định hướng của giảng viên, báo cáo không chèn nguyên cả khối mã nguồn dài gây loãng nội dung. Thay vào đó, báo cáo được tổ chức theo nguyên tắc: mỗi chương trước hết giải thích bài toán kỹ thuật, sau đó mới dùng hình và các trích đoạn mã ngắn để minh chứng. Các sơ đồ được dùng như công cụ chính để mô tả kiến trúc, flowchart và sequence diagram; mã nguồn chỉ được đưa vào khi cần giải thích chính xác cơ chế hoạt động của một module, một packet hoặc một trạng thái hệ thống.

## *1.5 Nguồn mã và khả năng đối chiếu*

Để bảo đảm tính minh bạch và khả năng kiểm chứng, báo cáo này đối chiếu trực tiếp với hai nhánh mã nguồn công khai của dự án:

- nhánh Sender: `https://github.com/dangtai111325/IOT_Application/tree/sender`
- nhánh Receiver: `https://github.com/dangtai111325/IOT_Application/tree/receiver`

Hai liên kết trên được dùng như nguồn tham chiếu gốc cho toàn bộ phần phân tích trong báo cáo: luồng khởi tạo, tổ chức module, giao thức TCP nội bộ, cơ chế đồng bộ cấu hình, tích hợp CoreIoT và các giới hạn kỹ thuật runtime. Cách đặt nguồn rõ ràng ngay ở đầu tài liệu giúp người đọc có thể đối chiếu từng luận điểm với mã nguồn tương ứng khi cần.




**Hình 1:** Kiến trúc tổng thể của hệ thống IoT Application theo mô hình nhiều sender, một receiver gateway và tầng cloud CoreIoT (demo hiện tại dùng hai sender).

**Phần II**

# Yêu cầu và định hướng thiết kế

# 2 Yêu cầu hệ thống và định hướng thiết kế

## 2.1 Định hướng từ đề bài môn học

Đề bài môn học yêu cầu sản phẩm phải có mức độ khác biệt đáng kể so với template gốc, đồng thời phải thể hiện rõ phần mở rộng về web server, TinyML và CoreIoT. Trong thực hiện của nhóm, phần khác biệt không chỉ nằm ở một chức năng riêng lẻ, mà còn nằm ở việc tái cấu trúc hệ thống thành hai lớp thiết bị sender và receiver với luồng điều khiển hai chiều. Cấu trúc này giúp sản phẩm vượt khỏi phạm vi một board duy nhất và tiệm cận tư duy thiết kế gateway thường gặp trong hệ thống IoT thực tế.

Về mặt báo cáo, điều này kéo theo yêu cầu phải mô tả rõ từng lớp: sender không chỉ đọc dữ liệu mà còn xử lý AI và phục vụ giao diện cục bộ; receiver không chỉ nhận dữ liệu mà còn làm điều phối mạng cục bộ, quản lý cấu hình và tích hợp cloud. Nếu chỉ trình bày báo cáo theo danh sách các file mã nguồn thì người đọc khó thấy được giá trị thật của đồ án. Vì vậy, cấu trúc báo cáo được xây dựng theo hướng: *bài toán – kiến trúc – giao thức – module – mã nguồn tiêu biểu – thực nghiệm*.

## 2.2 Yêu cầu chức năng

Các yêu cầu chức năng chính của hệ thống có thể tóm tắt như sau:

- Mỗi sender phải tạo được dữ liệu nhiệt độ, độ ẩm và vị trí GPS dưới dạng telemetry.
- Sender phải chạy được mô hình TinyML cục bộ và sinh ra `ai_score`, `ai_status`.
- Receiver phải nhận được telemetry từ nhiều sender qua TCP.
- Receiver phải phân biệt sender đã đăng ký và thiết bị lạ dựa trên `device_id`.
- Receiver phải đồng bộ cấu hình xuống sender, hỗ trợ thay đổi threshold và tên hiển thị.
- Cả sender và receiver phải có web UI cục bộ để cấu hình và quan sát trạng thái.
- Receiver phải đẩy được dữ liệu lên CoreIoT qua MQTT gateway và phản hồi RPC `getThreshold`/`setThreshold`.

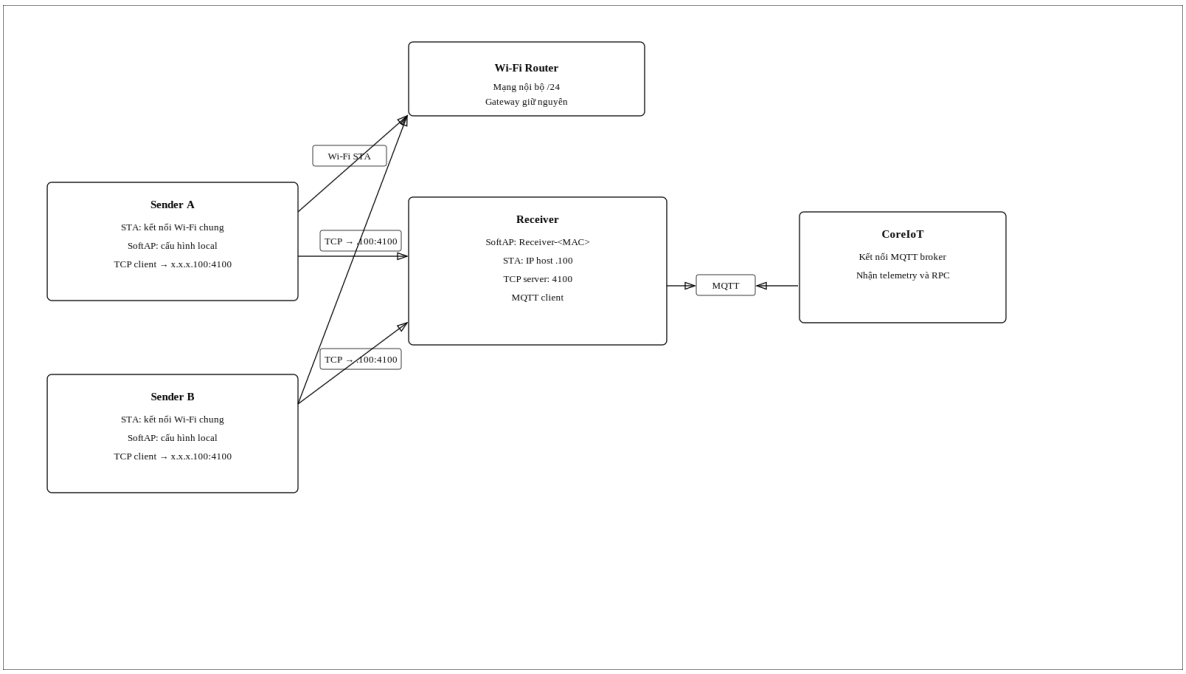
## 2.3 Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng, hệ thống còn cần đáp ứng một số yêu cầu phi chức năng quan trọng. Hệ thống phải dễ hiểu để phục vụ mục đích học tập và trình bày báo cáo. Mã nguồn cần được chia module hợp lý để dễ bảo trì. Giao thức giữa sender và receiver cần đủ đơn giản để dễ debug, nhưng vẫn phải thể hiện được cấu trúc chuẩn của packet trong hệ thống nhúng. Giao diện web phải đủ trực quan để người dùng không cần theo dõi serial log liên tục. Cuối cùng, thiết kế phải có khả năng mở rộng để trong tương lai có thể tăng số sender hoặc thay cảm biến

giả lập bằng cảm biến thật. Trong cấu hình hiện tại, receiver được thiết kế để quản lý tối đa 10 sender theo đúng giới hạn tài nguyên của firmware.

## *2.4 Lý do chọn kiến trúc nhiều sender – một receiver*

Nhóm chọn mô hình nhiều sender và một receiver vì đây là cấu trúc đủ nhỏ để kiểm soát trong phạm vi môn học, nhưng đủ lớn để thể hiện được tư duy thiết kế nhiều node. Cấu hình demo hiện tại dùng hai sender để giảm độ phức tạp khi trình diễn. Nếu chỉ có một sender, receiver gần như trở thành một điểm chuyển tiếp thụ động. Khi có từ hai sender trở lên, các vấn đề như quản lý danh tính thiết bị, phân rã trạng thái online/offline, xử lý đồng thời nhiều telemetry và đồng bộ cấu hình trở nên có ý nghĩa thật sự. Mô hình này cũng làm cho phần minh họa kiến trúc, luồng gói tin và dashboard trực quan hơn nhiều trong báo cáo.




**Hình 2:** Sơ đồ mạng nội bộ cho thấy receiver vừa phát SoftAP vừa kết nối Wi-Fi STA, trong khi sender suy ra địa chỉ receiver ở host .100.

**Phần III**

# Kiến trúc và thiết kế các khối

# 3 Kiến trúc tổng thể và phân rã hệ thống

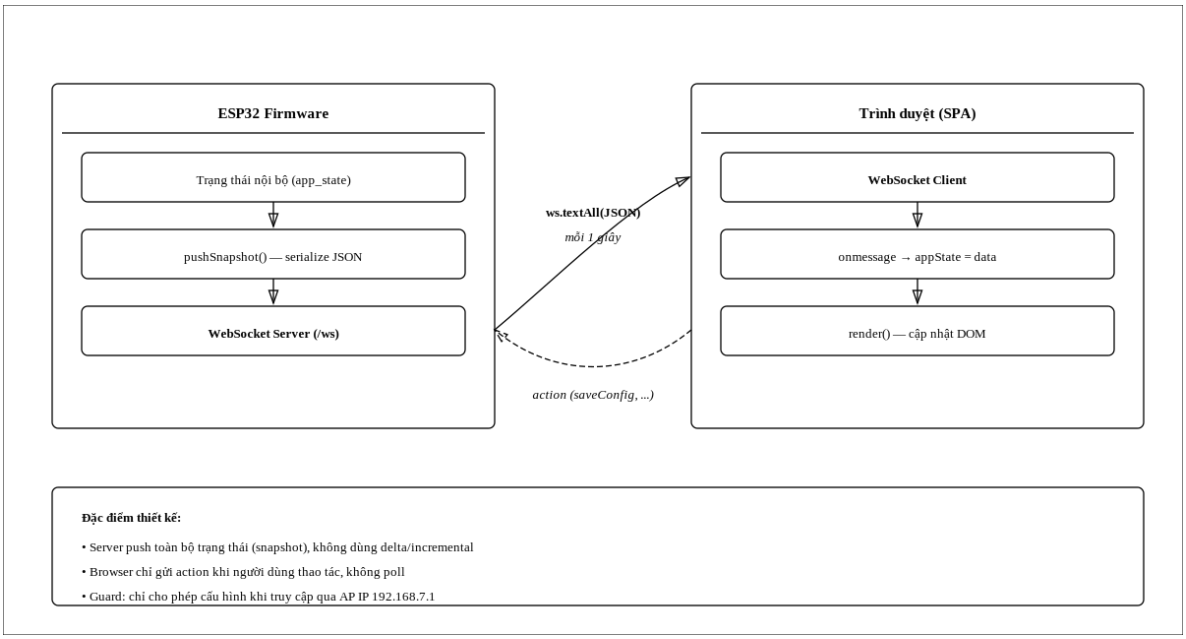
## 3.1 Bức tranh tổng quát của hệ thống

Hệ thống có thể được nhìn dưới ba lớp. Lớp thứ nhất là *edge sender*, nơi dữ liệu được tạo ra và được xử lý bằng TinyML. Lớp thứ hai là *receiver gateway*, nơi thu thập telemetry, quản lý cấu hình, tập trung web UI và làm đầu mối với cloud. Lớp thứ ba là *CoreIoT*, nơi hiển thị telemetry, lưu trữ thuộc tính và cung cấp RPC từ xa. Tách ba lớp như vậy giúp phân vai hệ thống rõ ràng: sender xử lý cục bộ và đo lường; receiver điều phối và đồng bộ; cloud chịu trách nhiệm trực quan hóa và điều khiển từ xa.

Điểm đáng chú ý là báo cáo không xem receiver là "board nhận dữ liệu" theo nghĩa hẹp. Trong kiến trúc này, receiver có đầy đủ vai trò của một gateway: nó có bảng trạng thái nhiều sender, có cơ chế persist cấu hình vào LittleFS, có logic retry khi gửi config, có cơ chế publish attribute khi threshold thay đổi, và có xử lý các RPC từ cloud. Đây là yếu tố tạo nên chiều sâu kỹ thuật của đề tài.

## 3.2 Mô hình data plane và control plane

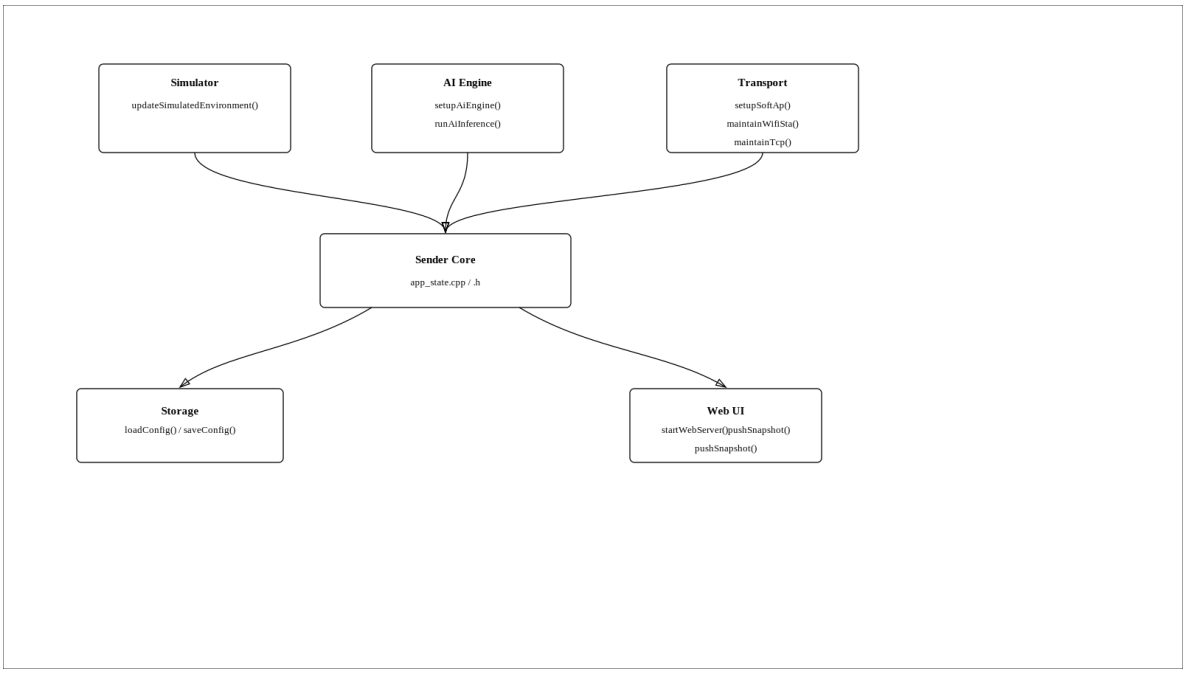
Một cách nhìn khác rất hữu ích cho báo cáo là tách riêng *data plane* và *control plane*. Data plane là tuyến đường đi lên của dữ liệu cảm biến: sender tạo telemetry, receiver parse và cập nhật `SenderState`, sau đó đẩy tiếp lên CoreIoT. Control plane là tuyến đường đi xuống của cấu hình: thao tác từ Web UI receiver hoặc RPC cloud được receiver chuyển thành packet `config`, sender áp dụng cấu hình và phản hồi `config_ack`. Khi tách như vậy, người đọc sẽ thấy hệ thống không chỉ thu thập dữ liệu mà còn thực hiện vòng phản hồi điều khiển khép kín.



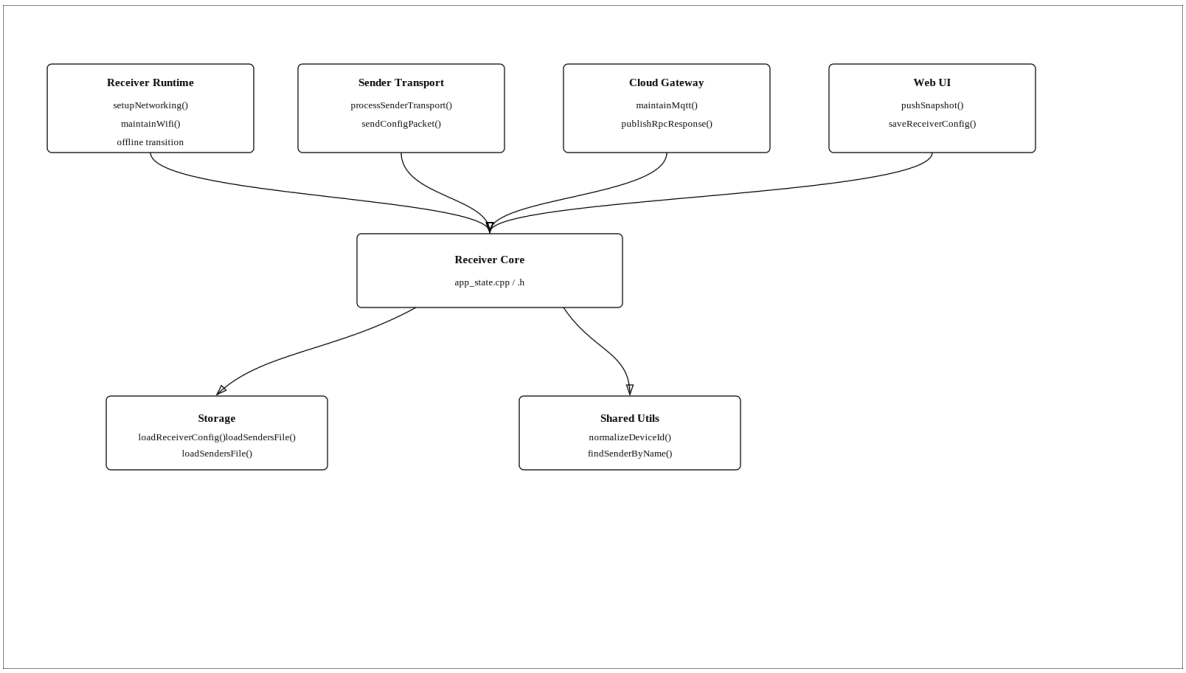

**Hình 3:** Sơ đồ tách *data plane* và *control plane* giúp nhận diện rõ hai loại luồng trong hệ thống.

## *3.3 Lợi ích của việc phân rã module*

Trong cả sender và receiver, mã nguồn được chia thành các module nhỏ thay vì dồn toàn bộ vào một file. Việc phân rã này mang lại ba lợi ích. Thứ nhất, báo cáo có thể giải thích từng khối chức năng độc lập mà không rơi vào tình trạng kể lại mã nguồn theo thứ tự dòng. Thứ hai, khi cần thay đổi một chức năng như giao thức mạng hoặc mô hình AI, người phát triển chỉ cần can thiệp vào module liên quan. Thứ ba, việc chia module phản ánh đúng tư duy thiết kế phần mềm nhúng hiện đại: dữ liệu trạng thái ở một nơi, logic mạng ở một nơi, logic UI ở một nơi, còn storage và cloud integration được đóng gói thành các lớp riêng.




**Hình 4:** Kiến trúc module của sender gồm các khối trạng thái, mô phỏng, AI, transport, lưu trữ và giao diện web.




**Hình 5:** Kiến trúc module của receiver gồm runtime, transport, cloud gateway, lưu trữ và giao diện web.

## *3.4 Bảng so sánh phần cứng và phần mềm của hai khối*

| Tiêu chí | Sender | Receiver |
|---|---|---|
| Vai trò chính | Nút biên tạo dữ liệu, chạy TinyML, gửi telemetry | Gateway trung tâm thu nhận, quản lý, đồng bộ cloud |
| Phần cứng mục tiêu | ESP32 DevKit (32 chân, board `esp32dev`) | ESP32-S3 (YOLO UNO ESP32-S3 trong cấu hình demo) |
| Chế độ Wi-Fi | AP + STA đồng thời (`WIFI_AP_STA`) | AP + STA đồng thời (`WIFI_AP_STA`) |
| Điểm truy cập cấu hình | AP cục bộ cho từng sender tại `192.168.7.1` | AP quản trị receiver tại `192.168.7.1` |
| Giao tiếp nội bộ | TCP client gửi JSON theo dòng lên receiver | TCP server lắng nghe cổng `4100` cho nhiều sender |
| Định danh thiết bị | `device_id` lấy từ MAC của sender | Quản lý sender theo khóa `device_id = MAC` |
| Lưu trữ cục bộ | `/sender.json` trên LittleFS | `/receiver.json`, `/senders.json` trên LittleFS |
| Tích hợp cloud | Không kết nối trực tiếp cloud | Kết nối CoreIoT qua MQTT gateway + RPC |

# 4 Thiết kế chi tiết khối Sender

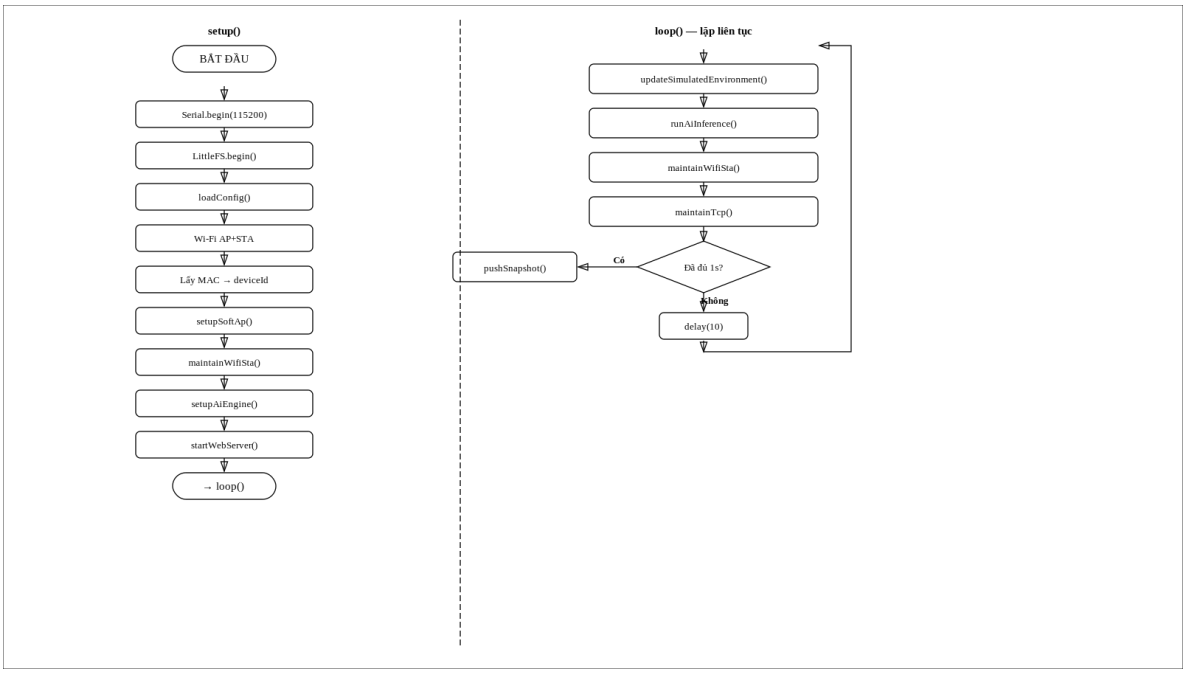
## *4.1 Vai trò của sender trong toàn hệ thống*

Mỗi sender được thiết kế như một node biên hoàn chỉnh. Nó không đơn thuần lấy dữ liệu rồi gửi đi, mà phải đồng thời thực hiện bốn trách nhiệm: sinh dữ liệu môi trường, chạy mô hình TinyML, duy trì kết nối mạng và cung cấp giao diện cục bộ cho người dùng. Cách thiết kế này phản ánh đúng tinh thần *edge computing*: xử lý tối thiểu phải diễn ra gần nơi phát sinh dữ liệu để giảm độ trễ và giảm phụ thuộc cloud.

Trong thực hiện của nhóm, sender sử dụng dữ liệu giả lập thay cho cảm biến thật. Tuy nhiên, từ góc nhìn kiến trúc, điều này không làm thay đổi vai trò của sender. Module `simulator` chỉ thay thế lớp driver cảm biến; các lớp AI, transport, storage và web UI vẫn vận hành y như khi có cảm biến thật. Chính nhờ vậy, báo cáo có thể tập trung vào luồng kỹ thuật cốt lõi mà không bị phụ thuộc vào một phần cứng cụ thể.

## *4.2 Luồng khởi tạo sender*

Khi khởi động, sender phải hoàn tất một chuỗi bước có tính phụ thuộc lẫn nhau. Trước hết, thiết bị khởi tạo serial để phục vụ debug. Sau đó LittleFS được mount để có thể đọc file cấu hình lưu từ những lần chạy trước. Tiếp theo, sender chuyển Wi-Fi sang chế độ `WIFI_AP_STA`, vì nó vừa cần phát AP cho cấu hình cục bộ, vừa phải kết nối lên mạng ngoài dưới vai trò STA. Sau khi lấy được MAC và sinh `deviceId`, sender tạo SSID local của riêng mình. Từ đây hệ thống lần lượt khởi tạo SoftAP, cố kết nối Wi-Fi ngoài, nạp mô hình TinyML và cuối cùng mở web server cục bộ.




**Hình 6:** Flowchart phần khởi tạo và vòng lặp sender. Sơ đồ cho thấy sender liên tục mô phỏng cảm biến, chạy AI và duy trì mạng.

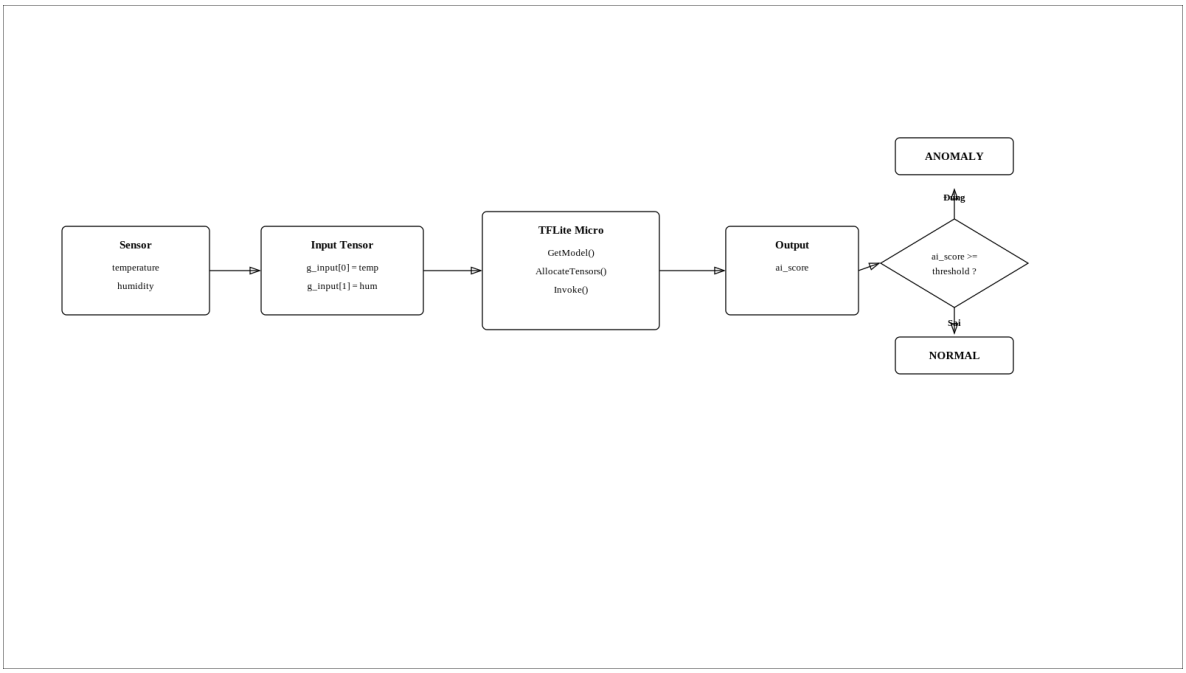
## 4.3 Mô phỏng dữ liệu sensor

Dữ liệu nhiệt độ và độ ẩm trên sender được mô phỏng theo kiểu *random walk có chặn biên*. Nghĩa là ở mỗi chu kỳ, giá trị mới được hình thành bằng cách cộng thêm một nhiễu ngẫu nhiên nhỏ vào giá trị cũ, sau đó chặn lại trong miền hợp lý. Cách mô phỏng này tốt hơn việc rút ngẫu nhiên độc lập mỗi lần, vì dữ liệu tạo ra có tính liên tục theo thời gian và nhìn giống dữ liệu môi trường thật hơn. GPS cũng được cập nhật theo cơ chế dịch chuyển nhẹ quanh một vị trí gốc, tạo ra quỹ đạo nhỏ đủ để minh họa dữ liệu vị trí.

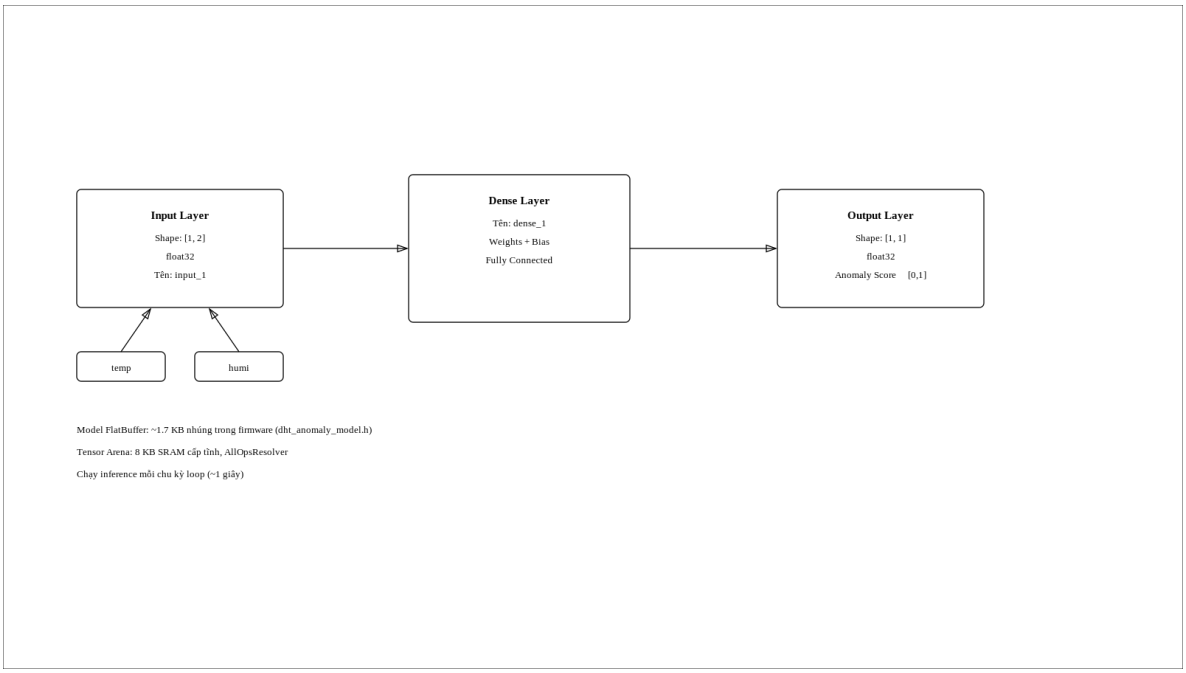
Về mặt báo cáo, cơ chế này đáng được trình bày vì nó giải thích tại sao dashboard của sender và receiver hiển thị dữ liệu biến thiên theo thời gian mà vẫn không nhảy loạn hoàn toàn. Dù đây chỉ là dữ liệu giả lập, cách tạo dữ liệu vẫn có dụng ý kỹ thuật để mô hình AI và giao diện trực quan hóa trông có ý nghĩa.

## 4.4 TinyML inference ở edge

Mô hình TinyML của sender nhận hai đầu vào là nhiệt độ và độ ẩm. Hai giá trị này được đưa vào tensor input, sau đó `MicroInterpreter` của TensorFlow Lite Micro thực hiện suy luận và tạo ra một giá trị `ai_score`. Điểm số này được chặn về khoảng $[0, 1]$ để bảo đảm tính ổn định khi hiển thị và so sánh. Nếu `ai_score` lớn hơn hoặc bằng `threshold` hiện tại, sender gắn nhãn `ANOMALY`; ngược lại, trạng thái là `NORMAL`. Vì sender là nơi tự thực hiện inference, receiver không cần tính lại trạng thái bất thường mà chỉ đóng vai trò tiếp nhận và phân phối thông tin.




**Hình 7:** Pipeline TinyML trên sender: từ dữ liệu mô phỏng sang tensor input, qua bộ thông dịch TFLite Micro, rồi trả ra điểm bất thường và nhãn trạng thái.




**Hình 8:** Mô hình logic của TFLite được sử dụng ở mức báo cáo: hai đầu vào đặc trưng và một đầu ra điểm bất thường.

## *4.5 Lớp mạng và sender transport*

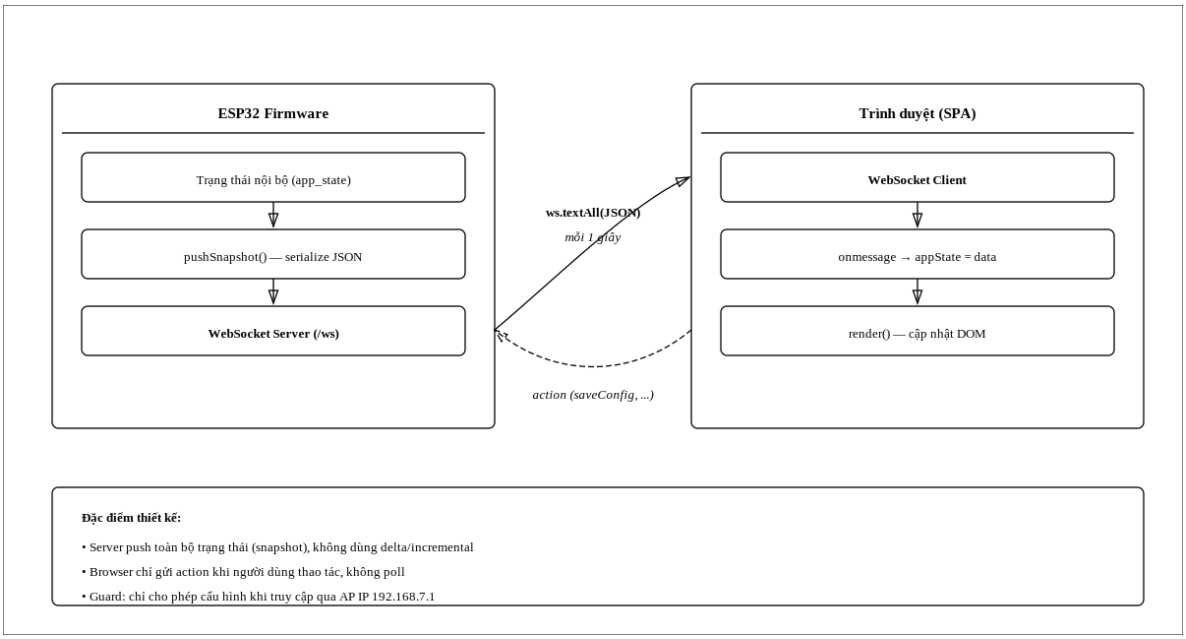
Sender transport là nơi hội tụ ba vai trò quan trọng: phát SoftAP, duy trì kết nối Wi-Fi STA và duy trì TCP client tới receiver. SoftAP cho phép người dùng truy cập giao diện web cục bộ ngay

cả khi sender chưa kết nối được Wi-Fi ngoài. Trong khi đó, lớp STA cho phép sender tham gia vào mạng chung với receiver và CoreIoT. Ở trên hai lớp Wi-Fi đó, sender mở một TCP client tới receiver. Đây là lựa chọn có chủ ý: với số lượng packet không lớn và cần sự đơn giản trong xử lý, dùng TCP với packet JSON theo dòng giúp dễ debug hơn nhiều so với việc tự thiết kế cơ chế truyền tin có xác nhận trên UDP.

Một chi tiết thiết kế khá đặc biệt là sender có thể tự suy ra địa chỉ receiver theo host `.100` nếu đang ở mạng `/24` và người dùng chưa cấu hình địa chỉ receiver bằng tay. Quy tắc này được cài đặt để giảm chi phí cấu hình trong môi trường thử nghiệm. Nếu người dùng nhập IP receiver thủ công thì giá trị này sẽ được ưu tiên sử dụng. Cách tổ chức này vừa giữ được tính tự động khi demo, vừa không làm mất khả năng can thiệp thủ công khi triển khai thực tế.

## *4.6 Giao diện web cục bộ của sender*

Sender có web UI riêng để người dùng quan sát trạng thái AI, mạng, cấu hình receiver và điều chỉnh threshold cục bộ. Giao diện này hoạt động nhờ web server phục vụ file tĩnh từ LittleFS và WebSocket `/ws` để đẩy snapshot thời gian thực. Cơ chế WebSocket giúp người dùng không phải refresh trang thủ công mà vẫn thấy được trạng thái mới sau mỗi chu kỳ. Về mặt thiết kế, đây là bước tiến đáng kể so với kiểu hệ nhúng chỉ dựa vào serial monitor.




**Hình 9:** Cơ chế WebSocket snapshot của sender và receiver, cho phép web UI nhận trạng thái thời gian thực mà không cần tải lại trang.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/sender_a_dashboard.png`

Mục tiêu ảnh: Ảnh chụp giao diện web của sender A đang hoạt động, gồm trạng thái kết nối, nhiệt độ, độ ẩm, AI score và threshold.

**Hình 10:** Ảnh thực nghiệm giao diện web của sender A.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/sender_b_dashboard.png`

Mục tiêu ảnh: Ảnh chụp giao diện web của sender B đang hoạt động. Nên thể hiện rõ sender B có mã thiết bị riêng và trạng thái độc lập với sender A.

**Hình 11:** Ảnh thực nghiệm giao diện web của sender B.

# 5 Thiết kế chi tiết khối Receiver

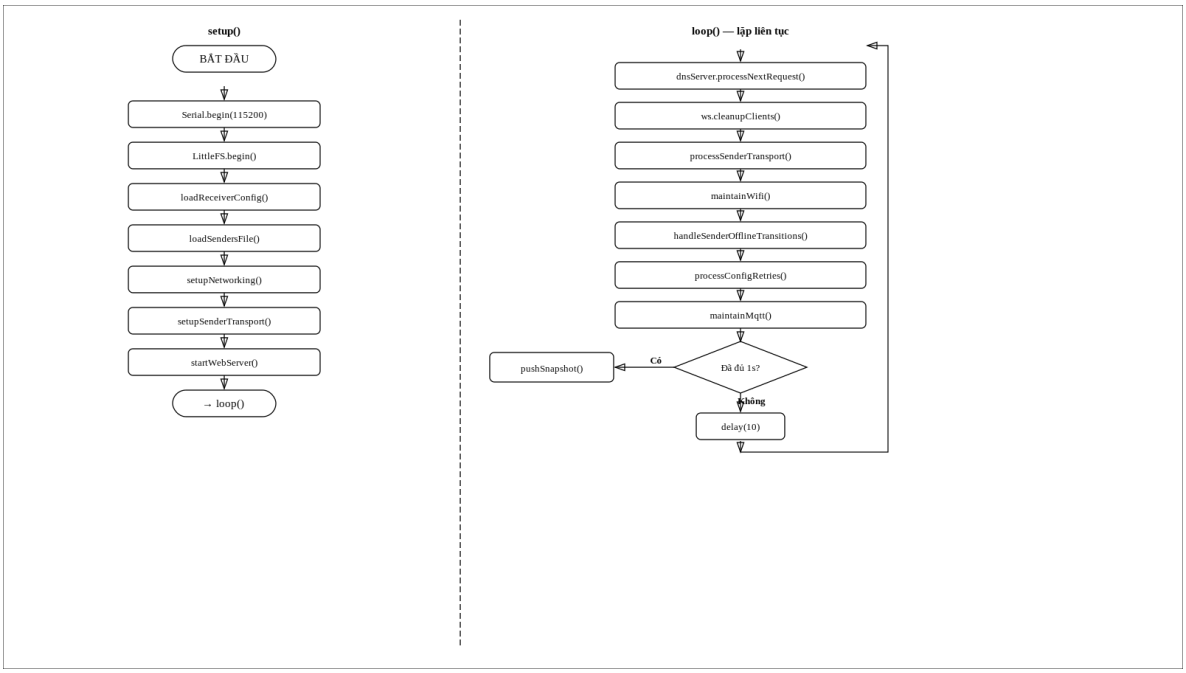
## *5.1 Receiver như một gateway thực thụ*

Receiver là khối quan trọng nhất của hệ thống. Nó không chỉ gom dữ liệu lại, mà còn nắm giữ toàn bộ logic điều phối của hệ thống. Receiver chịu trách nhiệm mở SoftAP quản trị, kết nối Wi-Fi ngoài, mở TCP server, theo dõi nhiều socket sender, lưu danh sách sender đã đăng ký, phân biệt thiết bị lạ, đẩy dữ liệu lên cloud, nhận RPC từ CoreIoT và chuyển các yêu cầu cấu hình tới sender tương ứng. Nói cách khác, receiver là nơi toàn bộ dữ liệu và điều khiển hội tụ.

Từ góc nhìn báo cáo, việc nhấn mạnh vai trò gateway là rất quan trọng. Nếu xem receiver như một thiết bị "thu và hiển thị", người đọc sẽ đánh giá thấp cấu trúc thật của hệ thống. Thực tế, các module của receiver cùng nhau tạo thành một phần mềm quản trị node biên thu nhỏ, với đầy đủ các khía cạnh networking, persistence, protocol parsing và cloud integration.

## 5.2 Luồng khởi tạo receiver

Quá trình khởi tạo receiver bắt đầu bằng việc mount LittleFS, nạp file cấu hình receiver và nạp danh sách sender đã đăng ký từ file `/senders.json`. Sau đó receiver thiết lập đồng thời SoftAP, DNS server và Wi-Fi STA. Việc khởi tạo TCP server được thực hiện riêng thành một bước để hệ thống sẵn sàng nhận sender mới bất cứ lúc nào. Cuối cùng receiver mở web server và WebSocket để chuẩn bị cho giao diện quản trị. Đây là trình tự hợp lý vì receiver phải có state và cấu hình trước khi có thể phục vụ UI hoặc xử lý sender.

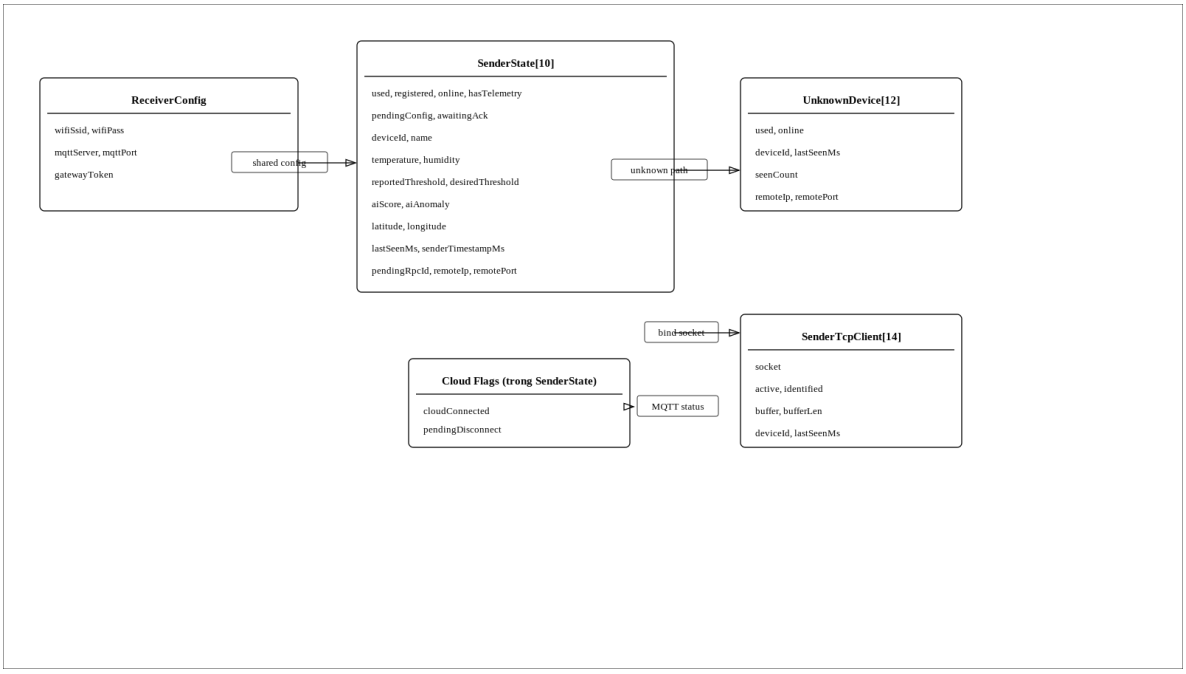



**Hình 12:** Flowchart phần khởi tạo và vòng lặp receiver. Sơ đồ cho thấy receiver đồng thời xử lý DNS, TCP, Wi-Fi, MQTT và snapshot cho UI.

## 5.3 Mô hình dữ liệu trung tâm của receiver

Receiver dùng bốn cấu trúc dữ liệu quan trọng. `ReceiverConfig` giữ thông tin Wi-Fi, server MQTT và token gateway. `SenderState` lưu toàn bộ trạng thái của sender đã đăng ký: online/offline, telemetry, threshold, cấu hình chờ đồng bộ, nonce cloud và địa chỉ remote. `UnknownDevice` dùng để theo dõi những sender đã gửi telemetry nhưng chưa nằm trong danh sách đăng ký. Cuối cùng `SenderTcpClient` biểu diễn từng socket TCP còn sống, kèm buffer để ghép các dòng JSON chưa hoàn tất.

Mô hình dữ liệu này là một điểm nổi bật vì nó cho thấy receiver không quản lý sender theo kiểu mảng giá trị đơn giản, mà phân chia rất rõ các lớp thông tin: cấu hình gateway, sender hợp lệ, thiết bị lạ và socket cấp thấp. Điều đó làm cho phần báo cáo trở nên mạch lạc hơn nhiều khi giải thích luồng hoạt động.

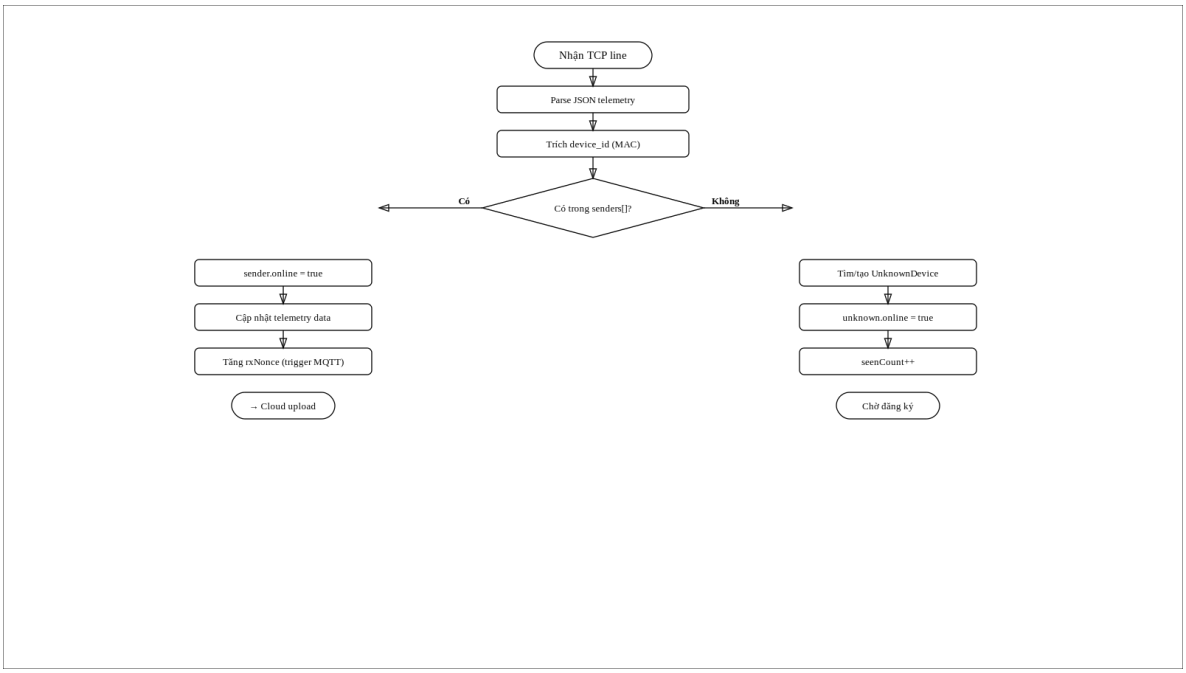



**Hình 13:** Mô hình dữ liệu trung tâm của receiver gồm cấu hình gateway, trạng thái sender đã đăng ký, thiết bị lạ và các kết nối TCP đang sống.

## *5.4 Known sender và unknown device*

Khi receiver nhận telemetry, việc đầu tiên không phải là đẩy dữ liệu lên cloud mà là kiểm tra `device_id`. Trong hệ thống này, `device_id` được gắn trực tiếp bằng địa chỉ MAC của sender để có khóa định danh ổn định xuyên suốt. Nếu `device_id` đã có trong bảng sender đăng ký thì telemetry được gắn vào `SenderState`. Nếu không, receiver chỉ đánh dấu thiết bị đó là `UnknownDevice`. Cách làm này có hai ý nghĩa. Thứ nhất, nó ngăn hệ thống tự động tin tưởng mọi thiết bị lạ xuất hiện trong mạng. Thứ hai, nó cho phép giao diện web hiển thị rõ rằng hiện có thiết bị chưa được quản lý chính thức, từ đó người dùng có thể chủ động đăng ký hoặc bỏ qua.

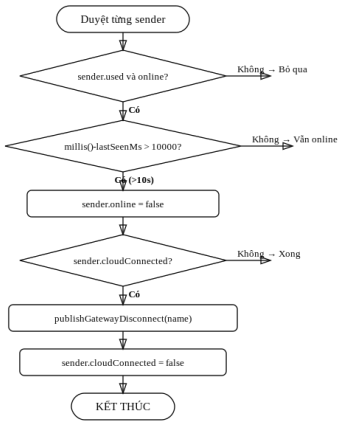
Đối với thiết bị lạ, receiver áp dụng nguyên tắc xử lý nhẹ: chỉ lưu các thông tin nhận diện và nhịp xuất hiện (`MAC/IP/lastSeen/seenCount`), không kích hoạt đầy đủ pipeline cloud telemetry. Quy tắc này giúp tiết kiệm tài nguyên và giữ hệ thống ổn định khi có lưu lượng không mong muốn.




**Hình 14:** Flowchart phân loại telemetry theo mã thiết bị: sender đã đăng ký được cập nhật vào bảng trạng thái, còn sender chưa đăng ký được đưa vào danh sách thiết bị lạ.

## *5.5 Cơ chế online/offline*

Receiver không phụ thuộc vào sự kiện ngắt kết nối TCP duy nhất để xác định sender offline. Thay vào đó, hệ thống dùng chiến lược timeout dựa trên `lastSeenMs`. Mỗi khi nhận telemetry hoặc ACK, receiver cập nhật `lastSeenMs`. Nếu quá `OFFLINE_TIMEOUT_MS` mà không thấy dữ liệu mới, sender được chuyển sang trạng thái offline. Với sender đã từng kết nối cloud, receiver còn tạo `pendingDisconnect` để gửi sự kiện disconnect lên CoreIoT. Cơ chế này phản ánh cách hệ thống phân tán thường quản lý *liveness*: trạng thái online là kết quả của heartbeat gần nhất, không phải một cờ tĩnh.




**Hình 15:** Flowchart chuyển trạng thái online/offline của sender trên receiver dựa vào timeout.

## *5.6 Web UI quản trị của receiver*

Giao diện web của receiver có hai vai trò. Vai trò thứ nhất là quan sát trạng thái tập trung của toàn hệ thống: thông tin receiver, cloud status, danh sách sender, unknown devices và đồng bộ cấu hình. Vai trò thứ hai là cho phép quản trị: thêm sender, gán tên, đặt threshold mong muốn, xóa sender, lưu cấu hình Wi-Fi hoặc CoreIoT. Việc gom các chức năng này vào receiver là hợp lý vì receiver là node duy nhất nhìn thấy đầy đủ cả hệ thống.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/receiver_dashboard.png`

Mục tiêu ảnh: Ảnh chụp giao diện receiver đang hiển thị đồng thời hai sender, trạng thái cloud và danh sách unknown devices (nếu có).

**Hình 16:** Ảnh thực nghiệm giao diện quản trị của receiver.

**Phần IV**

# Giao thức, luồng điều khiển và tích hợp cloud

# 6 Thiết kế giao thức truyền thông

## 6.1 Lý do chọn TCP nội bộ

Một quyết định thiết kế đáng phân tích là việc dùng TCP cho đường sender – receiver. Trong những bài demo nhỏ, người ta thường dùng giao thức càng đơn giản càng tốt, hoặc thậm chí gửi broadcast không có xác nhận. Tuy nhiên, trong hệ thống này nhóm cần một đường truyền ổn định, dễ debug, và thuận tiện để triển khai cơ chế cấu hình hai chiều. Dùng TCP mang lại lợi ích ở chỗ gói tin được đảm bảo thứ tự, giảm gánh nặng tự xử lý truyền lại ở tầng ứng dụng. Thêm vào đó, khi packet ở dạng JSON theo dòng, việc đọc log và phân tích sai lệch khi debug trở nên rất trực quan.

TCP nội bộ cũng phù hợp với receiver đóng vai trò server và sender đóng vai trò client. Kiến trúc này giúp sender chủ động kết nối về receiver sau khi đã có Wi-Fi STA, trong khi receiver luôn lắng nghe ở một port cố định. Nhờ đó, khi sender rơi mạng rồi quay lại, chính sender là nơi tái tạo kết nối mà không cần receiver quét lại mạng.

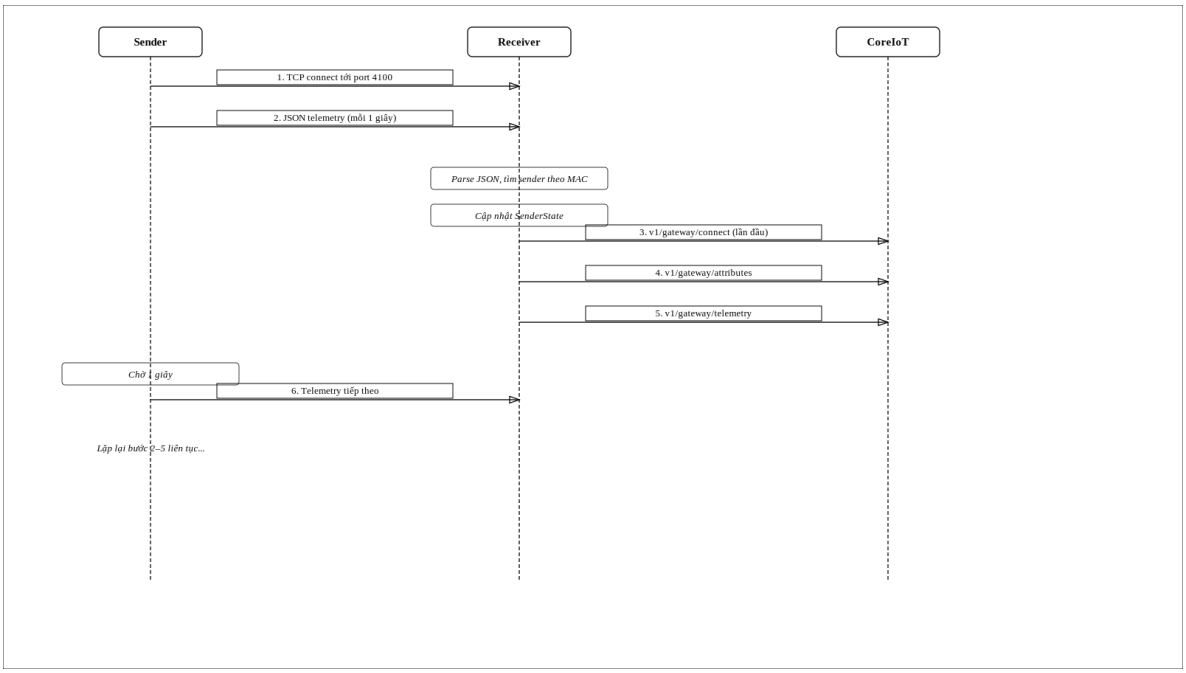
## 6.2 Định dạng packet TCP

Trên đường TCP, hệ thống sử dụng ba loại packet chính: `telemetry`, `config` và `config_ack`. Packet `telemetry` là gói sender gửi lên receiver theo chu kỳ. Packet `config` do receiver gửi xuống sender khi cần thay đổi threshold hoặc metadata. Packet `config_ack` do sender trả lại sau khi áp dụng hoặc từ chối cấu hình. Thiết kế chỉ dùng ba loại gói đã đủ để xây dựng một vòng điều khiển khép kín đơn giản nhưng rõ ràng. Toàn bộ các gói này đi qua TCP server của receiver trên cổng `4100`.

| Loại message | Hướng | Các trường chính | Mô tả |
|---|---|---|---|
| telemetry | Sender → Receiver | device_id, temperature, humidity, threshold, ai_score, ai_status, latitude, longitude, timestamp | Dữ liệu cảm biến + kết quả AI, mỗi 1s |
| config | Receiver → Sender | device_id, name, threshold, receiver_ip, receiver_port, config_version | Cấu hình mới từ Receiver/RPC |
| config_ack | Sender → Receiver | device_id, success, applied_threshold, config_version, message | Phản hồi sau khi áp dụng config |

*Mỗi message là 1 dòng JSON kết thúc bằng ký tự \n. Kết nối TCP persistent.*

**Hình 17:** Bảng tổng hợp ba loại message TCP và các trường dữ liệu chính được trao đổi giữa sender và receiver.




**Hình 18:** Sequence diagram mô tả nhiều sender gửi telemetry lên receiver, receiver cập nhật trạng thái và tiếp tục publish lên cloud.

## 6.3 Chi tiết packet telemetry

Packet `telemetry` chứa `device_id`, nhiệt độ, độ ẩm, threshold hiện tại trên sender, `ai_score`, `ai_status`, tọa độ GPS và timestamp. Việc gửi cả threshold hiện tại lên receiver có một ý nghĩa tinh tế: nếu sender thay đổi threshold cục bộ bằng giao diện web của chính nó, receiver vẫn có thể phát hiện sự thay đổi này và học lại giá trị mới khi không có phiên cấu hình nào đang chờ. Điều này làm cho hệ thống linh hoạt hơn thay vì bắt buộc mọi thay đổi đều phải đi qua gateway.

## 6.4 Chi tiết packet config và ACK

Packet `config` do receiver sinh ra từ `SenderState` gồm `device_id`, tên sender, threshold mong muốn, địa chỉ IP/port của receiver và `configVersion`. Sender chỉ áp dụng gói nếu `device_id` trùng với chính nó. Sau khi lưu lại cấu hình, sender trả về `config_ack` với `success` và `appliedThreshold`. Nhờ ACK, receiver không phải suy đoán việc cấu hình đã thành công hay chưa. Đây là lý do hệ thống có thể hỗ trợ retry một cách minh bạch.

## 6.5 Giao thức MQTT với CoreIoT

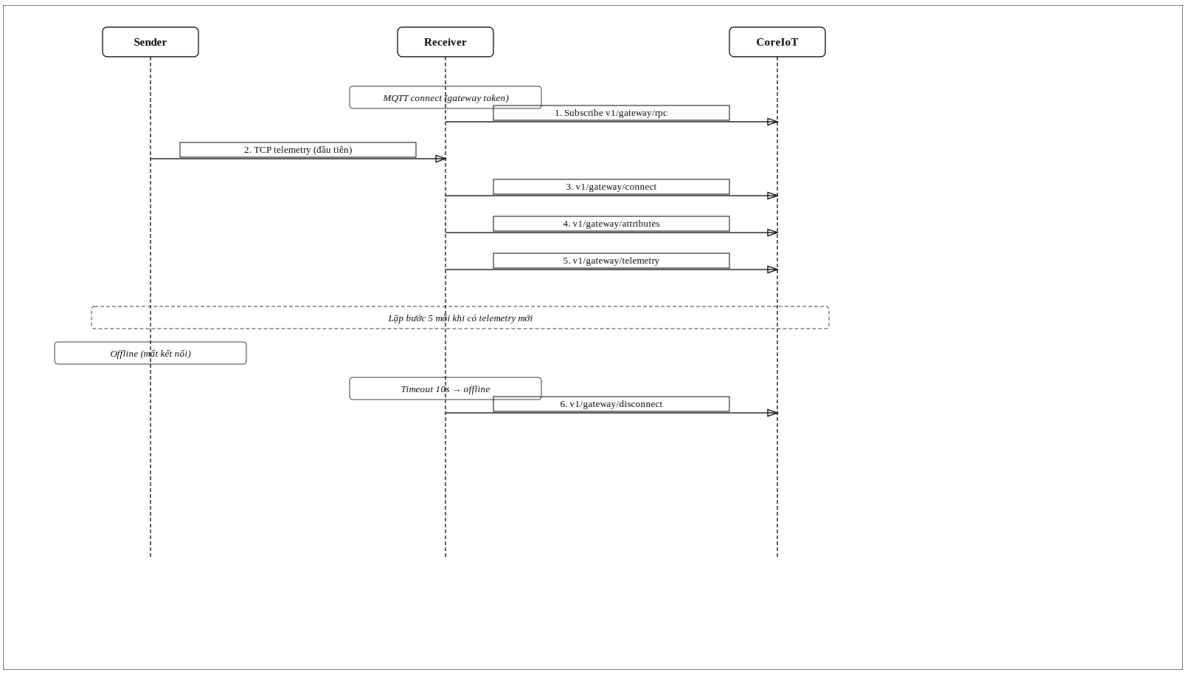
Với CoreIoT, receiver dùng mô hình MQTT gateway. Điều này có nghĩa receiver không mở một MQTT session riêng cho từng sender mà dùng một client duy nhất rồi đại diện cho nhiều sender qua các topic gateway. Cách làm này rất đúng với vai trò gateway: sender không cần

biết gì về token cloud, không phải tự kết nối Internet trực tiếp với broker, và toàn bộ kiểm soát cloud được tập trung ở receiver. Từ đó, việc thay đổi cấu hình cloud chỉ cần thực hiện ở một nơi.

| Topic | Hướng | Payload (ví dụ) | Mô tả |
|---|---|---|---|
| v1/gateway/connect | Pub → | {"device":"Sender01","type":"dangtai"} | Đăng ký thiết bị con |
| v1/gateway/disconnect | Pub → | {"device":"Sender01"} | Báo ngắt kết nối |
| v1/gateway/telemetry | Pub → | {"Sender01":[{"ts":...,"values":{...}}]} | Gửi telemetry |
| v1/gateway/attributes | Pub → | {"Sender01":{"device_id":...,"threshold":0.5}} | Gửi attributes |
| v1/gateway/rpc | ← Sub | {"device":"Sender01","data":{"method":"set..."}} | Nhận lệnh RPC |

*Receiver đóng vai trò MQTT Gateway, kết nối bằng gateway_token.*

**Hình 19:** Bảng tổng hợp các topic MQTT gateway mà receiver dùng để làm việc với CoreIoT.




**Hình 20:** Sequence diagram mô tả receiver đại diện sender khi gửi connect, telemetry, attribute và xử lý RPC qua MQTT gateway.

# 7 Thiết kế luồng điều khiển cấu hình và RPC

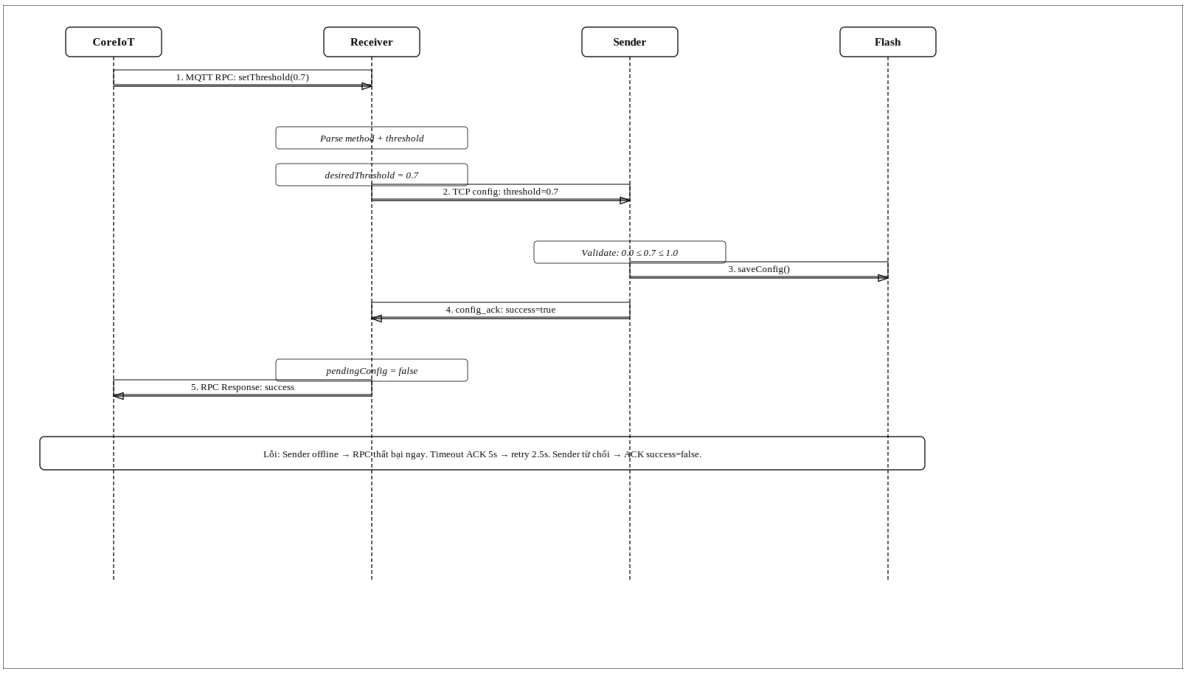
## *7.1 Tại sao cần đồng bộ cấu hình hai chiều*

Trong hệ thống nhiều node, việc cấu hình nằm rải rác ở từng sender sẽ nhanh chóng khó quản lý. Nếu người vận hành thay đổi threshold trên một sender qua web local, thay sender khác qua cloud, rồi muốn kiểm tra trạng thái cuối cùng ở receiver, hệ thống cần một mô hình đồng bộ đủ rõ để tránh mâu thuẫn. Đó là lý do receiver duy trì đồng thời

`reportedThreshold`, `desiredThreshold`, `pendingConfig`, `awaitingAck`, `configVersion` và `ackedConfigVersion`. Bộ biến này là xương sống của control plane.

## 7.2 Luồng đổi threshold từ receiver UI

Khi người dùng thao tác trên receiver UI để thay đổi threshold của một sender, receiver trước tiên kiểm tra tính hợp lệ của `device_id`, tên hiển thị và ngưỡng mới. Sau đó receiver ghi `desiredThreshold`, tăng `configVersion`, bật `pendingConfig` và nếu sender đang online thì lập tức gửi `config`. Sender nhận gói, áp dụng ngưỡng mới, lưu xuống LittleFS rồi trả `config_ack`. Chỉ khi nhận được ACK hợp lệ, receiver mới đánh dấu quá trình đồng bộ hoàn tất. Cơ chế này giúp giao diện receiver phản ánh đúng rằng thay đổi đã thực sự tới được sender, thay vì chỉ là ý định cấu hình.



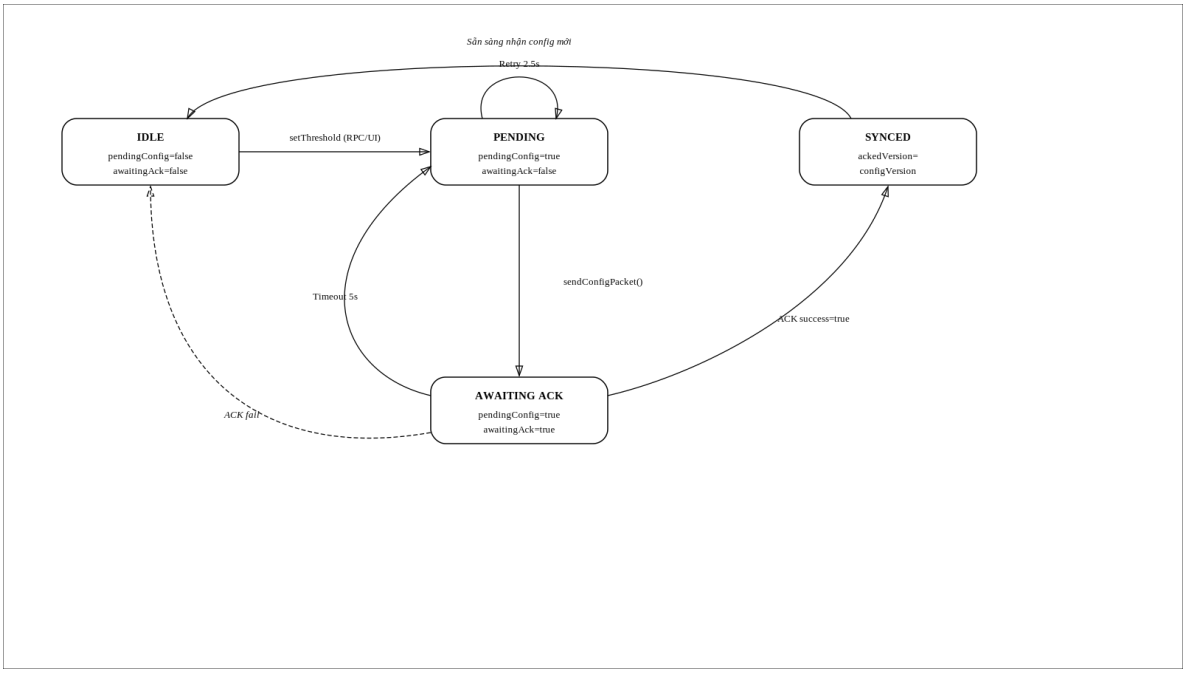

**Hình 21:** Sequence diagram mô tả quá trình receiver gửi packet cấu hình xuống sender và nhận lại gói xác nhận. Cùng cơ chế này cũng được dùng khi có RPC từ cloud.

## 7.3 Luồng đổi threshold từ RPC cloud

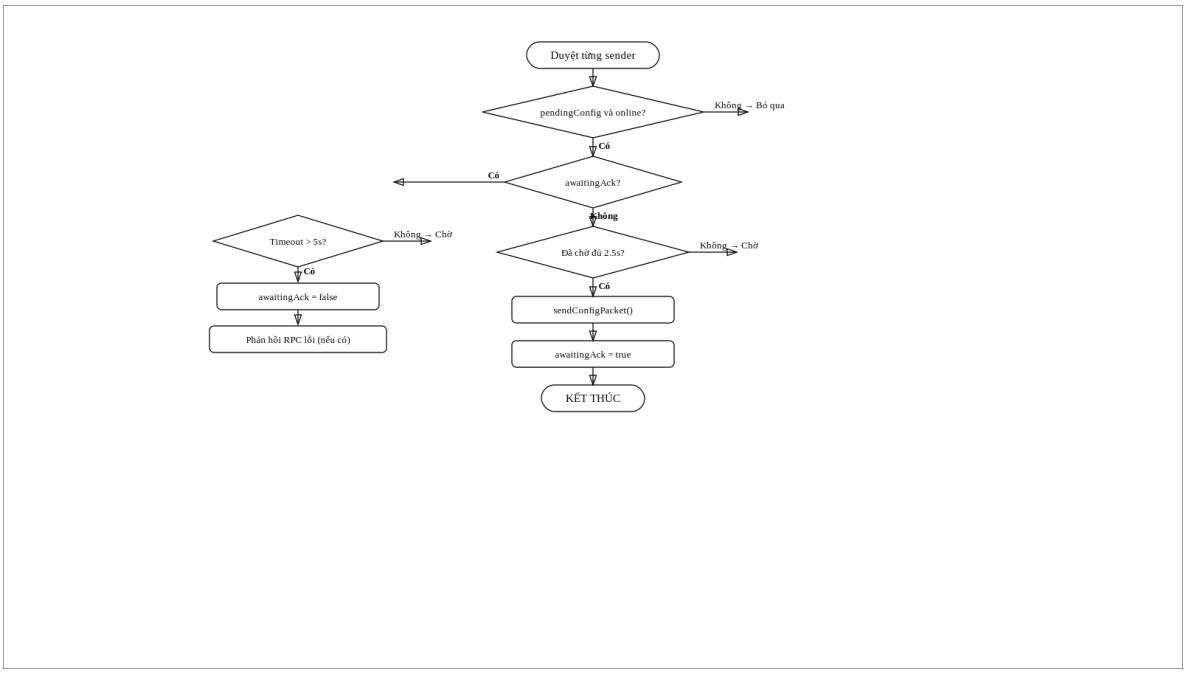
Khi CoreIoT gửi RPC `setThreshold`, receiver không cập nhật sender ngay ở cloud mà chuyển đổi RPC đó thành một phiên cấu hình nội bộ. Nghĩa là receiver parse phương thức, tìm sender theo tên, cập nhật `desiredThreshold`, rồi gọi chính cơ chế `sendConfigPacket()` như ở Web UI local. Khi sender ACK thành công, receiver mới gửi RPC response ngược lại cloud. Với cách này, cloud chỉ đóng vai trò nguồn phát lệnh; còn receiver vẫn là nơi duy nhất có quyền điều phối sender. Đây là thiết kế đúng với mô hình gateway và giúp mọi đường điều khiển đều hội tụ về một lớp logic thống nhất.

## *7.4 Retry và timeout*

Nếu sender chưa ACK sau một khoảng thời gian định trước, receiver không xem đó là thất bại vĩnh viễn ngay lập tức. Thay vào đó, `processConfigRetries()` sẽ kiểm tra các sender đang có `pendingConfig`. Nếu sender vẫn online nhưng `awaitingAck` đã quá timeout, hệ thống hạ cờ `awaitingAck` và có thể thử gửi lại. Nếu yêu cầu xuất phát từ RPC cloud, receiver đồng thời tạo phản hồi lỗi phù hợp để platform biết rằng sender không xác nhận kịp thời. Đây là ví dụ rõ ràng cho thấy một hệ thống nhúng nhỏ vẫn có thể tổ chức logic đồng bộ có trạng thái một cách bài bản.




**Hình 22:** State machine đồng bộ cấu hình giữa receiver và sender.




**Hình 23:** Flowchart cơ chế retry cấu hình của receiver khi sender chưa ACK trong thời gian cho phép.

# 8 Tích hợp CoreIoT và vai trò của cloud

## *8.1 Receiver như một MQTT gateway*

Receiver dùng đúng tinh thần *gateway* khi kết nối CoreIoT. Khi một sender online và đã đăng ký, receiver gửi `v1/gateway/connect` để mở phiên thiết bị trên cloud. Ngay sau đó, receiver có thể gửi `v1/gateway/attributes` để đồng bộ metadata như `device_id`, `receiver_ip` và threshold. Tiếp theo, mỗi khi có telemetry mới, receiver publish dữ liệu lên `v1/gateway/telemetry`. Nếu sender offline hoặc bị xóa, receiver gửi `v1/gateway/disconnect`. Nhờ cơ chế này, cloud nhìn thấy mỗi sender như một thực thể riêng, dù về mặt vật lý chỉ receiver mới là thiết bị trực tiếp mở phiên MQTT.

Việc tách `attributes` và `telemetry` cũng là một quyết định quan trọng. Dữ liệu biến thiên nhanh (nhiệt độ, độ ẩm, ai_score, trạng thái AI, GPS) được gửi theo telemetry; dữ liệu định danh/cấu hình (`device_id`, threshold, thông tin receiver) được gửi theo attributes và chỉ cập nhật khi cần. Cách tách này giúp dashboard dễ đọc hơn và giảm phát sinh dữ liệu lặp không cần thiết.

## *8.2 Giá trị của tầng cloud đối với hệ thống*

Trong đồ án này, cloud không được dùng để thay thế web UI cục bộ mà được xem là tầng giám sát và điều khiển từ xa. Điều này rất quan trọng. Web UI cục bộ giúp cấu hình nhanh trong môi trường thử nghiệm, nhất là khi chưa có Internet hoặc khi cần truy cập AP tại chỗ. Cloud lại có

vai trò lịch sử hóa, trực quan hóa tập trung và điều khiển từ xa. Cách tách hai vai trò như vậy giúp kiến trúc cân bằng: local UI phục vụ vận hành tại chỗ, cloud phục vụ quản trị từ xa.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/coreiot_dashboard.png`

Mục tiêu ảnh: Ảnh chụp dashboard hoặc timeline thiết bị trên CoreIoT. Nên thể hiện rõ nhiều sender xuất hiện như các device riêng biệt, có biểu đồ telemetry và thuộc tính threshold.

**Hình 24:** Ảnh thực nghiệm dashboard CoreIoT sau khi receiver publish dữ liệu.

## *8.3 Ý nghĩa của việc hỗ trợ RPC*

Hỗ trợ RPC `getThreshold`/`setThreshold` giúp hệ thống tiến gần hơn tới mô hình điều khiển từ xa thực sự. Thay vì cloud chỉ là nơi xem dữ liệu, người vận hành có thể gửi một yêu cầu điều chỉnh threshold từ dashboard. Yêu cầu đó đi qua receiver và được xác nhận bởi sender. Vì có vòng ACK nên RPC phản ánh chính xác trạng thái thực của thiết bị, chứ không chỉ là lệnh gửi đi mà không biết đã được áp dụng hay chưa.

**Phần V**

# Ràng buộc, độ tin cậy và phân tích mã nguồn

# 9 Ràng buộc kỹ thuật và quyết định thiết kế

## 9.1 Bảng hằng số hệ thống và ý nghĩa vận hành

Để báo cáo có tính kỹ thuật và có thể kiểm chứng bằng mã nguồn, bảng sau tổng hợp các hằng số quan trọng đang điều khiển hành vi runtime. Các giá trị cụ thể có thể thay đổi theo phiên bản code, nhưng ý nghĩa kỹ thuật của từng nhóm hằng số cần được giữ ổn định.

| Nhóm hằng số | Ví dụ trong code | Ý nghĩa kỹ thuật |
|---|---|---|
| Tần suất cảm biến / AI | `SENSOR_INTERVAL_MS` | Chu kỳ cập nhật dữ liệu môi trường và suy luận TinyML ở sender. |
| Tần suất gửi telemetry | `TELEMETRY_INTERVAL_MS` | Chu kỳ đẩy dữ liệu từ sender sang receiver; ảnh hưởng trực tiếp độ mượt dashboard. |
| Tần suất snapshot UI | `SNAPSHOT_INTERVAL_MS` | Chu kỳ phát snapshot qua WebSocket; cân bằng giữa realtime và tải CPU/mạng. |
| Timeout online/offline | `OFFLINE_TIMEOUT_MS` | Ngưỡng quyết định sender bị đánh dấu offline khi không còn dữ liệu mới. |
| Retry cấu hình | `CONFIG_RETRY_MS` | Khoảng chờ trước khi gửi lại packet cấu hình nếu chưa nhận ACK. |
| Timeout ACK cấu hình | `CONFIG_ACK_TIMEOUT_MS` | Ngưỡng kết luận một phiên đồng bộ cấu hình bị quá hạn. |
| Retry Wi-Fi / MQTT | `WIFI_RETRY_MS`, `MQTT_RETRY_MS` | Tần suất thử kết nối lại hạ tầng mạng và cloud. |
| Giới hạn số sender/client | `MAX_SENDERS`, `MAX_TCP_CLIENTS` | Ràng buộc tài nguyên của receiver; tránh vượt năng lực RAM/socket. |
| Giới hạn xử lý TCP theo chu kỳ | `TCP_MAX_BYTES_PER_CYCLE`, `TCP_MAX_LINES_PER_CYCLE` | Chống một sender chiếm toàn bộ vòng lặp, đảm bảo công bằng cho nhiều sender. |

## 9.2 Trade-off của các quyết định kiến trúc

- **TCP line-delimited JSON thay vì giao thức nhị phân**: dễ debug và dễ trình bày trong báo cáo; đánh đổi là overhead chuỗi lớn hơn.
- **Receiver giữ vai trò gateway duy nhất**: giúp tập trung điều phối, đồng bộ trạng thái và cloud integration; đánh đổi là receiver trở thành điểm trọng yếu duy nhất.
- **TinyML chạy ở sender**: phản hồi bất thường nhanh tại edge; đánh đổi là firmware sender phức tạp hơn và phải quản lý tài nguyên suy luận.
- **AP+STA đồng thời**: thuận tiện cấu hình cục bộ khi demo; đánh đổi là cần xử lý cẩn thận

trạng thái mạng để tránh xung đột luồng.

– **Cơ chế ACK + retry cho cấu hình**: tăng độ tin cậy điều khiển; đánh đổi là state machine phức tạp hơn so với gửi một chiều.

# 10 Tình huống lỗi, cơ chế phòng vệ và bằng chứng kỹ thuật

## *10.1 Các tình huống lỗi điển hình*

1) Sender mất kết nối tạm thời hoặc reboot giữa chừng.
2) Receiver nhận packet JSON lỗi định dạng hoặc thiếu trường.
3) RPC cloud gửi xuống khi sender đang offline.
4) ACK cấu hình không về đúng hạn do nghẽn mạng nội bộ.
5) Thiết bị lạ gửi telemetry nhưng chưa đăng ký trong danh sách sender chính thức.

## *10.2 Cơ chế phòng vệ trong mã nguồn*

– Parse JSON theo từng dòng và bỏ qua gói lỗi để không làm hỏng vòng lặp chính.
– Tách `known sender` và `unknown device` để tránh tin tưởng thiết bị lạ.
– Chỉ xác nhận đồng bộ cấu hình thành công khi đã nhận `config_ack` hợp lệ.
– Dùng timeout + retry có kiểm soát thay vì gửi lặp vô hạn.
– Giới hạn số byte/số dòng mỗi chu kỳ xử lý TCP để bảo vệ tính công bằng giữa nhiều sender.

## *10.3 Bảng đối chiếu luận điểm và bằng chứng mã nguồn*

| Luận điểm kỹ thuật | Bằng chứng trong mã nguồn | Ý nghĩa |
|---|---|---|
| Sender suy luận bất thường tại edge | Hàm `runAiInference()` ở sender | Giảm phụ thuộc cloud, phản hồi trạng thái nhanh tại nguồn dữ liệu. |
| Receiver điều phối cấu hình hai chiều | `sendConfigPacket()`, `applyConfigAck()` | Đảm bảo lệnh cấu hình được xác nhận thay vì gửi một chiều. |
| Quản lý liveness bài bản | `handleSenderOfflineTraining()` | Trạng thái online/offline dựa trên timeout, không phụ thuộc cảm tính. |

| Luận điểm kỹ thuật | Bằng chứng trong mã nguồn | Ý nghĩa |
|---|---|---|
| Tách sender hợp lệ và thiết bị lạ | Nhánh xử lý known/unknown trong transport | Tăng an toàn vận hành và rõ ràng nghiệp vụ. |
| Cloud chỉ nhìn thấy sender qua gateway | Nhóm hàm MQTT gateway connect/telemetry/rpc | Mô hình triển khai đúng kiến trúc gateway thực tế. |

## *10.4 Ghi chú hình cần chèn ở chương này*

**GHI CHÚ CHÈN HÌNH MỚI**

Mã hình: `fig:todo-state-config`
Đường dẫn dự kiến: `assets/HINH_RANG_BUOC_01.png`

Mục tiêu hình: Sơ đồ state machine đồng bộ cấu hình: pending, awaiting ACK, timeout, retry, success/fail.

**Hình 25:** Sơ đồ state machine đồng bộ cấu hình: pending, awaiting ACK, timeout, retry, success/fail.

**GHI CHÚ CHÈN HÌNH MỚI**

Mã hình: `fig:todo-rpc-offline`
Đường dẫn dự kiến: `assets/HINH_RANG_BUOC_02.png`

Mục tiêu hình: Sequence xử lý lỗi RPC khi sender offline và phản hồi lại cloud.

**Hình 26:** Sequence xử lý lỗi RPC khi sender offline và phản hồi lại cloud.

**GHI CHÚ CHÈN HÌNH MỚI**

Mã hình: `fig:todo-slot-fairness`
Đường dẫn dự kiến: `assets/HINH_RANG_BUOC_03.png`
Mục tiêu hình: Flowchart phân bổ slot TCP nhiều sender và cơ chế chống starvation.

**Hình 27:** Flowchart phân bổ slot TCP nhiều sender và cơ chế chống starvation.

# 11 Phân tích mã nguồn theo module và theo dòng ý nghĩa

## *11.1 Tổ chức phân tích mã nguồn*

Để tránh tình trạng báo cáo biến thành nơi sao chép mã nguồn, phần này chỉ chọn các khối mã có ý nghĩa cao nhất để phân tích. Mỗi trích đoạn mã đều đi kèm ba phần: vai trò của hàm trong hệ thống, giải thích logic xử lý theo bước, và ý nghĩa thiết kế. Cách trình bày này cho phép người đọc hiểu *vì sao hàm tồn tại* và *vì sao nó được viết như vậy*, thay vì chỉ biết hàm đó làm gì.

## *11.2 Trích đoạn 1: vòng lặp chính của sender*

Đoạn mã dưới đây thể hiện rất rõ triết lý của sender: cập nhật môi trường, chạy AI, duy trì mạng và đẩy snapshot. Điều đáng chú ý là mọi việc được tổ chức theo kiểu cooperative loop chứ không dùng thêm task nền mới.

```
void appLoop()
{
    updateSimulatedEnvironment();
    runAiInference();
    maintainWifiSta();
    maintainTcp();

    if (millis() - g_lastSnapshotAt >= SNAPSHOT_INTERVAL_MS)
    {
        g_lastSnapshotAt = millis();
        pushSnapshot();
    }
```

```

    delay(10);
}
```

*Đoạn mã cho thấy sender thực hiện tuần tự mô phỏng dữ liệu, suy luận AI, giữ kết nối và cập nhật giao diện web.*

Điểm mạnh của cấu trúc này là đơn giản và dễ quan sát. Mỗi vòng lặp đều thực hiện các tác vụ cốt lõi, trong đó các hàm con tự kiểm soát bằng timestamp nội bộ để không chạy quá dày. Cách tổ chức như vậy giúp giảm độ phức tạp so với việc tách quá nhiều task FreeRTOS trong một đồ án môn học, nhưng vẫn đủ rõ ràng để mở rộng sau này.

## 11.3 Trích đoạn 2: suy luận TinyML trên sender

```
void runAiInference()
{
    if (g_interpreter == nullptr || g_input == nullptr || g_output
   == nullptr)
    {
        g_aiScore = NAN;
        g_aiStatus = "AI ERROR";
        return;
    }

    g_input->data.f[0] = g_temperature;
    g_input->data.f[1] = g_humidity;

    if (g_interpreter->Invoke() != kTfLiteOk)
    {
        g_aiScore = NAN;
        g_aiStatus = "AI ERROR";
        return;
    }

    g_aiScore = g_output->data.f[0];
    g_aiStatus = g_aiScore >= g_config.threshold ? "ANOMALY" : "
   NORMAL";
}
```

*Hàm suy luận TinyML cho thấy đầu vào của mô hình là nhiệt độ và độ ẩm, còn đầu ra là `ai_score` được so sánh với threshold.*

Trong đoạn mã này, điều quan trọng không phải là lời gọi `Invoke()` mà là cách sender tự chịu

trách nhiệm về tính hợp lệ của inference. Nếu interpreter chưa sẵn sàng hoặc trả về lỗi, sender không cố tiếp tục với giá trị sai mà chuyển hẳn sang trạng thái `AI ERROR`. Đây là lựa chọn hợp lý vì giúp dashboard và receiver phân biệt giữa dữ liệu hợp lệ và lỗi mô hình.

## 11.4 Trích đoạn 3: gửi telemetry qua TCP

```
void sendTelemetry()
{
    JsonDocument doc;
    doc["type"] = "telemetry";
    doc["device_id"] = g_deviceId;
    doc["temperature"] = g_temperature;
    doc["humidity"] = g_humidity;
    doc["threshold"] = g_config.threshold;
    doc["ai_score"] = g_aiScore;
    doc["ai_status"] = g_aiStatus;
    doc["latitude"] = g_latitude;
    doc["longitude"] = g_longitude;
    doc["timestamp"] = senderTimestampMs();

    String payload;
    serializeJson(doc, payload);
    payload += '\n';
    g_tcpClient.print(payload);
}
```

*Sender dùng JSON newline-delimited để đóng gói telemetry, rất dễ đọc và dễ debug trong quá trình thử nghiệm.*

Thiết kế newline-delimited JSON có ưu điểm rõ rệt ở đồ án này. Một mặt, receiver có thể đọc theo từng ký tự và tách gói bằng `\n`. Mặt khác, khi cần quan sát packet thật qua serial hoặc công cụ mạng, người phát triển có thể đọc ngay nội dung mà không cần giải mã định dạng nhị phân.

## 11.5 Trích đoạn 4: xử lý telemetry ở receiver

```
void applyTelemetry(int slotIndex, const SenderTelemetryPacket &
    packet, const WiFiClient &client)
{
     const int senderIndex = findSenderByDeviceId(String(packet.
    deviceId));
     if (senderIndex < 0)
     {
```

```
        markUnknownHeartbeat(String(packet.deviceId), client);
        return;
    }

    SenderState &sender = g_senders[senderIndex];
    sender.online = true;
    sender.hasTelemetry = true;
    sender.lastSeenMs = millis();
    sender.temperature = packet.temperature;
    sender.humidity = packet.humidity;
    sender.reportedThreshold = packet.threshold;
    sender.aiScore = packet.aiScore;
    sender.aiAnomaly = packet.aiAnomaly;
}
```

*Receiver không mặc định chấp nhận mọi thiết bị. Chỉ sender đã đăng ký mới được cập nhật vào `SenderState`; sender lạ sẽ rơi vào nhánh `UnknownDevice`.*

Đây là một trong những đoạn mã quan trọng nhất của toàn hệ thống. Nó minh họa đúng tinh thần gateway: trước khi lưu telemetry, receiver luôn ràng buộc thiết bị vào `device_id`. Nhờ bước kiểm tra này, hệ thống tránh được việc một node lạ xuất hiện trong mạng lại ngay lập tức được xem là thiết bị hợp lệ trên cloud.

## 11.6 Trích đoạn 5: gửi cấu hình từ receiver xuống sender

```
bool sendConfigPacket(int index, uint32_t rpcId)
{
    JsonDocument doc;
    doc["type"] = "config";
    doc["device_id"] = sender.deviceId;
    doc["name"] = sender.name;
    doc["threshold"] = sender.desiredThreshold;
    doc["receiver_ip"] = WiFi.localIP().toString();
    doc["receiver_port"] = LOCAL_TCP_PORT;
    doc["config_version"] = sender.configVersion;

    String payload;
    serializeJson(doc, payload);
    payload += '\n';
    client.socket.print(payload);

    sender.pendingConfig = true;
    sender.awaitingAck = true;
```

```
    sender.configSentAtMs = millis();
    return true;
}
```

*Receiver gửi packet `config` kèm `configVersion`, rồi chuyển sender sang trạng thái chờ ACK.*

Điểm quan trọng ở đây là sau khi gửi cấu hình, receiver không đánh dấu đồng bộ thành công ngay lập tức. Hệ thống chủ động chuyển sang trạng thái `awaitingAck`. Điều này rất quan trọng trong các hệ thống có điều khiển từ xa, bởi vì *lệnh đã gửi* không đồng nghĩa với *thiết bị đã áp dụng*. Cấu trúc state machine trong hệ thống được xây trên nguyên lý đó.

## 11.7 Trích đoạn 6: publish RPC response lên CoreIoT

```
void publishRpcResponse(const char *deviceName, uint32_t requestId,
                        bool success, const char *message, float
    threshold)
{
    JsonDocument doc;
    doc["device"] = deviceName;
    doc["id"] = requestId;
    JsonObject data = doc["data"].to<JsonObject>();
    data["success"] = success;
    data["message"] = message;
    if (!std::isnan(threshold))
    {
        data["threshold"] = threshold;
    }
    mqttPublish(MQTT_GATEWAY_RPC_TOPIC, doc);
}
```

*RPC response được trả ngược lên CoreIoT sau khi receiver đã biết rõ kết quả cuối cùng từ sender.*

Đoạn mã này làm rõ vai trò cầu nối của receiver. Cloud không trực tiếp biết sender nội bộ đã áp dụng cấu hình hay chưa. Chỉ receiver mới tổng hợp được điều đó dựa trên ACK từ sender, rồi phát hành một RPC response có trạng thái thành công hoặc thất bại. Đó chính là giá trị của gateway trong thiết kế này.

**Phần VI**

# Giao diện, đánh giá và kết luận

# 12 Thiết kế web UI và trải nghiệm vận hành

## *12.1 Web UI sender*

Web UI của sender phục vụ ba nhóm nhu cầu. Một là theo dõi trạng thái cảm biến và AI theo thời gian thực. Hai là quan sát tình trạng kết nối receiver, bao gồm địa chỉ IP đã cấu hình và trạng thái TCP hiện tại. Ba là thay đổi nhanh các tham số cấu hình như Wi-Fi, địa chỉ receiver và threshold. Vì dùng WebSocket, sender có thể đẩy snapshot tới nhiều client đồng thời mà không cần người dùng bấm tải lại trang.

## *12.2 Web UI receiver*

Giao diện receiver là nơi người vận hành có cái nhìn tập trung về toàn hệ thống. Tại đây có thể thấy rõ receiver đã kết nối cloud hay chưa, địa chỉ mạng hiện tại, danh sách sender đang online, trạng thái AI của từng sender, các sender đang có cấu hình chờ đồng bộ và những thiết bị lạ chưa đăng ký. Đối với một đồ án môn học, giao diện như vậy có giá trị lớn vì nó biến các trạng thái nội bộ thành thông tin quan sát được, giúp việc trình diễn và giải thích dễ dàng hơn nhiều.

## *12.3 Ảnh thực nghiệm giao diện và log*

Phần này cần được nhóm bổ sung bằng ảnh thật khi chuẩn bị bản nộp cuối cùng. Trong giai đoạn soạn thảo tài liệu nguồn, các vị trí ảnh đã được đánh dấu sẵn để khi có screenshot chỉ cần chép file vào đúng đường dẫn là tài liệu sẽ tự động hiển thị.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/receiver_sender_table.png`

Mục tiêu ảnh: Ảnh chụp receiver hiển thị đồng thời nhiều sender trong bảng trạng thái, bao gồm tên, mã thiết bị, trạng thái online, threshold và AI status.

**Hình 28:** Ảnh bảng trạng thái sender trên receiver.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/serial_log_sender_a.png`

Mục tiêu ảnh: Ảnh chụp serial log của sender A khi khởi động, kết nối Wi-Fi, kết nối receiver và gửi telemetry.

**Hình 29:** Ảnh log serial của sender A.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/serial_log_receiver.png`

Mục tiêu ảnh: Ảnh chụp serial log của receiver khi nhận telemetry, cập nhật sender và kết nối MQTT.

**Hình 30:** Ảnh log serial của receiver.

# 13 Kịch bản thực nghiệm và kế hoạch đánh giá

## *13.1 Mục tiêu đánh giá*

Để báo cáo có giá trị kỹ thuật, phần thực nghiệm không nên chỉ dừng ở việc nói rằng "hệ thống chạy được". Thay vào đó, cần chứng minh được bốn loại hành vi. Thứ nhất là hành vi bình thường: nhiều sender cùng hoạt động, gửi telemetry đều đặn, receiver hiển thị đúng và cloud nhận dữ liệu. Thứ hai là hành vi cấu hình: khi đổi threshold từ receiver UI hoặc từ cloud RPC, sender nhận được packet `config`, áp dụng và ACK. Thứ ba là hành vi lỗi và khôi phục: sender

mất kết nối rồi online lại, receiver cập nhật online/offline đúng như mong đợi. Thứ tư là hành vi AI: khi ngưỡng thay đổi hoặc dữ liệu biến động, `ai_status` trên sender và receiver thay đổi tương ứng.

## *13.2 Danh sách test case đề xuất*

| TC | Mục tiêu | Bước thực hiện | Kết quả mong đợi |
|---|---|---|---|
| TC1 | Khởi động hệ thống bình thường | Bật nguồn receiver, sau đó bật các sender trong cấu hình demo và chờ khởi tạo hoàn tất | Các sender online trên receiver, có telemetry, cloud kết nối ổn định |
| TC2 | Gửi telemetry liên tục | Giữ hệ thống chạy ít nhất 3–5 phút | Receiver nhận dữ liệu tuần tự, cloud có timeline mới, UI cập nhật real-time |
| TC3 | Đổi threshold từ receiver UI | Mở receiver UI, đổi threshold sender A và lưu cấu hình | Sender A nhận `config`, cập nhật threshold, trả ACK, receiver bỏ cờ syncPending |
| TC4 | Đổi threshold từ cloud RPC | Gửi RPC `setThreshold` cho sender B từ CoreIoT | Receiver nhận RPC, gửi packet xuống sender B, sender B ACK, cloud nhận RPC response |
| TC5 | Sender offline | Tắt sender A hoặc ngắt Wi-Fi của sender A | Receiver đánh dấu offline sau timeout và nếu cần publish disconnect |

| TC | Mục tiêu | Bước thực hiện | Kết quả mong đợi |
|---|---|---|---|
| TC6 | Sender reconnect | Bật lại sender A sau TC5 | Receiver nhận telemetry mới và sender A trở lại trạng thái online |
| TC7 | AI anomaly | Giảm threshold xuống giá trị thấp hoặc điều chỉnh dữ liệu mô phỏng | `ai_status` chuyển sang `ANOMALY` trên sender, receiver và cloud |
| TC8 | Unknown device | Cho một sender chưa đăng ký gửi về receiver | Receiver không thêm vào sender chính thức mà đưa vào danh sách unknown device |

## *13.3 Chỉ số quan sát cần ghi lại*

Khi thực hiện các test case, nhóm nên ghi nhận ít nhất các thông tin sau: thời gian boot của sender và receiver; thời gian từ lúc sender online đến lúc receiver hiển thị telemetry đầu tiên; thời gian từ khi gửi RPC đến khi sender ACK; số lần retry cấu hình nếu có; số lần trạng thái offline bị kích hoạt; và một số ảnh chụp thể hiện thay đổi của `ai_status` khi điều chỉnh threshold. Những chỉ số này không nhất thiết phải chính xác đến mức benchmark công nghiệp, nhưng chúng giúp phần đánh giá bớt cảm tính và tăng tính thuyết phục của báo cáo.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/testcase_threshold_change.png`

Mục tiêu ảnh: Ảnh chuỗi thử nghiệm đổi threshold thành công. Nên gồm ít nhất hai trạng thái trước và sau thay đổi.

**Hình 31:** Ảnh minh họa test case đổi threshold.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/testcase_sender_offline.png`

Mục tiêu ảnh: Ảnh thể hiện sender offline rồi reconnect. Có thể là ghép hai ảnh UI hoặc một ảnh log.

**Hình 32:** Ảnh minh họa test case sender offline rồi reconnect.

## *13.4 Đánh giá ưu điểm của hệ thống*

Sau khi hệ thống chạy ổn định ở các test case trên, có thể rút ra một số ưu điểm kỹ thuật. Trước hết, kiến trúc tách sender và receiver làm cho hệ thống có tính mô đun tốt: sender tập trung vào sensing và AI, receiver tập trung vào gateway và cloud. Thứ hai, TinyML được đặt ở sender nên thời gian quyết định bất thường rất ngắn và receiver không phải gánh thêm tính toán. Thứ ba, control plane có ACK và retry, nhờ đó những thay đổi threshold không còn là hành động "gửi rồi hy vọng" mà có xác nhận rõ ràng. Thứ tư, việc có cả Web UI local và cloud giúp hệ thống vừa thuận tiện khi demo tại chỗ vừa có giá trị thực tế khi quan sát từ xa.

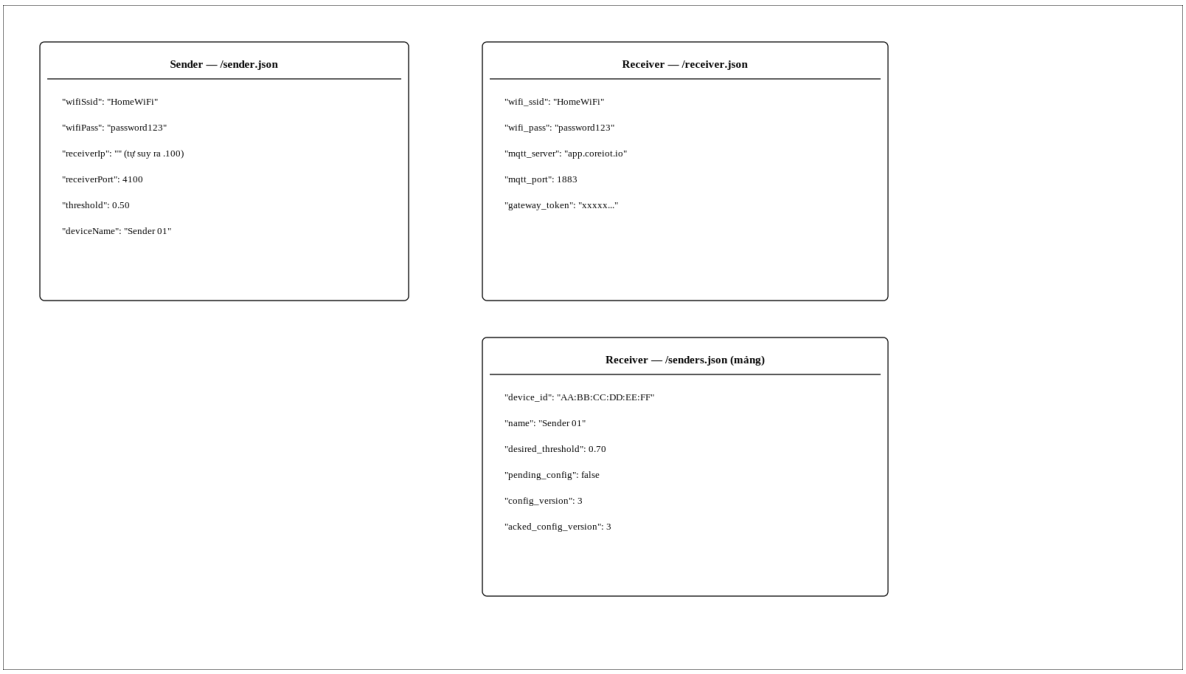
### *13.5 Hạn chế hiện tại*

Dù đạt được mục tiêu học thuật chính, hệ thống vẫn còn một số hạn chế cần nêu thẳng trong báo cáo. Thứ nhất, dữ liệu sensor hiện được mô phỏng nên chưa phản ánh đầy đủ nhiễu và sai số của cảm biến thật. Thứ hai, timestamp trên sender được sinh từ `millis()` cộng với một epoch giả định nên không phải đồng hồ thực chuẩn NTP hay RTC. Thứ ba, cơ chế suy luận địa chỉ receiver theo host `.100` chỉ tối ưu cho subnet `/24`; nếu chuyển sang cấu trúc mạng khác, logic này cần điều chỉnh. Thứ tư, TCP nội bộ chưa có lớp bảo mật và xác thực mạnh; trong môi trường sản phẩm thực tế cần thêm TLS hoặc cơ chế bảo vệ cao hơn.

## 14 Danh sách ghi chú hình minh họa cần chèn

### *14.1 Nhóm hình kiến trúc và mạng*

Mục này là danh sách ghi chú hình cần chèn để hoàn thiện bản nộp cuối. Mỗi khung ghi chú đều chứa đường dẫn dự kiến và mục tiêu kỹ thuật của hình, giúp nhóm bổ sung hình nhanh mà vẫn giữ đúng mạch phân tích.

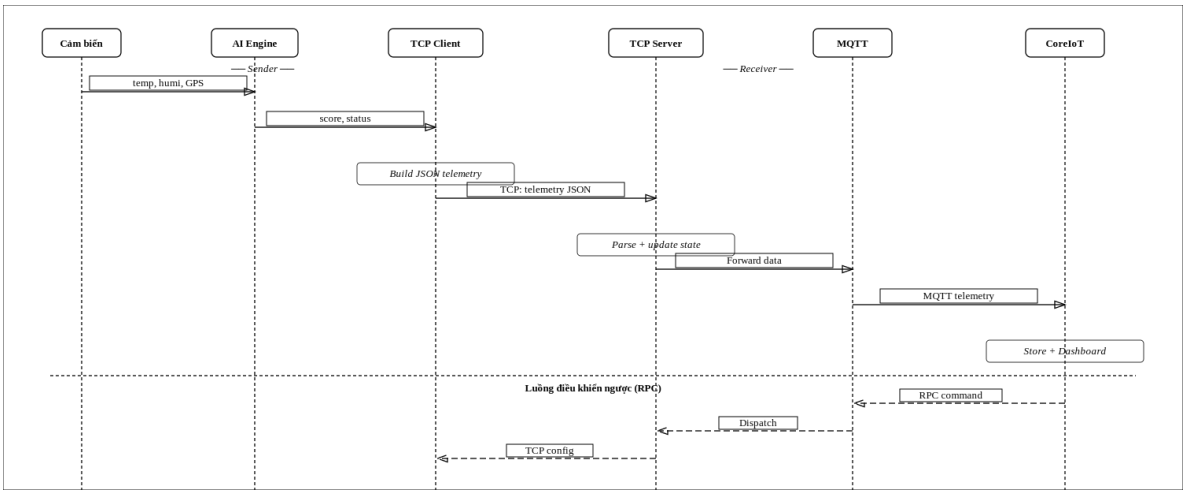



**Hình 33:** Cấu trúc lưu trữ LittleFS của sender và receiver. Hình này giúp người đọc hiểu rõ những tệp nào là cấu hình và những tệp nào dùng để phục vụ web UI.

| Topic | Hướng | Payload (ví dụ) | Mô tả |
|---|---|---|---|
| v1/gateway/connect | Pub → | {"device":"Sender01","type":"dangtai"} | Đăng ký thiết bị con |
| v1/gateway/disconnect | Pub → | {"device":"Sender01"} | Báo ngắt kết nối |
| v1/gateway/telemetry | Pub → | {"Sender01":[{"ts":...,"values":{...}}]} | Gửi telemetry |
| v1/gateway/attributes | Pub → | {"Sender01":{"device_id":...,"threshold":0.5}} | Gửi attributes |
| v1/gateway/rpc | ← Sub | {"device":"Sender01","data":{"method":"set..."}} | Nhận lệnh RPC |

*Receiver đóng vai trò MQTT Gateway, kết nối bằng gateway_token.*

**Hình 34:** Bảng topic MQTT dùng bởi receiver khi giao tiếp với CoreIoT.




**Hình 35:** Sequence end-to-end tổng hợp cả dữ liệu đi lên, điều khiển đi xuống và snapshot tới browser.

# 15 Bàn luận kỹ thuật và hướng phát triển

## *15.1 Vì sao kiến trúc này phù hợp cho môn học*

Một yêu cầu khó của học phần là hệ thống phải vừa đủ phức tạp để thể hiện năng lực thiết kế, nhưng vẫn đủ nhỏ để một nhóm sinh viên có thể hoàn thiện trong thời gian giới hạn. Mô hình nhiều sender – một receiver thỏa mãn điều kiện đó; bản demo hiện tại triển khai hai sender để trình diễn ổn định. Nó không quá đơn giản như một board đơn lẻ, nhưng cũng chưa phức tạp tới mức cần thêm message broker riêng hay database ngoài. Nhờ vậy, đồ án trở thành một điểm cân bằng hợp lý giữa chiều sâu kiến trúc và khả năng hoàn thành.

## *15.2 Khả năng mở rộng trong tương lai*

Từ nền tảng hiện tại, có thể mở rộng hệ thống theo nhiều hướng. Hướng thứ nhất là thay dữ liệu mô phỏng bằng cảm biến thật như DHT20, BME280 hoặc cảm biến khí, từ đó đánh giá lại sự ổn định của TinyML khi đầu vào có nhiễu thực. Hướng thứ hai là mở rộng số sender và đánh

giá khả năng quản lý của receiver khi số socket tăng lên. Hướng thứ ba là bổ sung OTA update hoặc đồng bộ firmware để receiver không chỉ là gateway dữ liệu mà còn là điểm điều phối vòng đời thiết bị. Hướng thứ tư là tăng tính bảo mật bằng mã hóa, xác thực hai chiều hoặc quản lý khóa. Hướng thứ năm là làm giàu dashboard cloud để hỗ trợ rule engine và cảnh báo phức tạp hơn.

### *15.3 Khả năng áp dụng thực tế*

Dù đây là một đồ án học phần, kiến trúc tổng quát của hệ thống có thể áp dụng vào nhiều bài toán thật với mức điều chỉnh vừa phải. Ví dụ, trong nhà kính thông minh, các sender có thể là các cụm đo nhiệt độ – độ ẩm – ánh sáng, còn receiver là gateway trung tâm đặt gần router. Trong phòng thí nghiệm, các sender có thể thu các đại lượng từ các khu vực khác nhau, còn receiver là nơi tập trung cấu hình và hiển thị. Nếu thay mô hình TinyML hiện tại bằng mô hình huấn luyện cho một bài toán cụ thể hơn, hệ thống có thể trở thành nền tảng prototype cho cảnh báo sớm ngay tại edge.

## 16 Kết luận

Báo cáo đã trình bày đầy đủ quá trình phân tích và hiện thực hệ thống *IoT Application* theo mô hình nhiều sender một receiver, trong đó sender chịu trách nhiệm mô phỏng cảm biến, thực hiện TinyML tại edge và cung cấp giao diện cục bộ, còn receiver đóng vai trò gateway trung tâm kết nối sender với CoreIoT. Điểm cốt lõi của đồ án không nằm ở số lượng tính năng rời rạc, mà nằm ở cách tổ chức các tính năng thành một kiến trúc thống nhất: dữ liệu đi lên qua data plane, cấu hình đi xuống qua control plane, và toàn bộ quá trình đều có trạng thái rõ ràng để theo dõi, xác nhận và phục hồi.

Thông qua việc phân tích kiến trúc, giao thức, module và mã nguồn tiêu biểu, báo cáo cho thấy hệ thống đã vượt qua phạm vi một bài demo cảm biến đơn giản để tiến gần hơn tới một mô hình IoT gateway thực thụ. Cách triển khai này phù hợp với yêu cầu của môn học, đồng thời là nền tảng tốt để tiếp tục mở rộng ở các đồ án lớn hơn trong tương lai.

**Phần VII**

# Phụ lục

# 1 Phụ lục A: Trích đoạn mã nguồn tiêu biểu và diễn giải ngắn

Phụ lục này tập hợp các đoạn mã nguồn tiêu biểu để người đọc đối chiếu nhanh với phần phân tích trong nội dung chính.

## *A.1 Khởi tạo sender*

```
void appSetup()
{
    Serial.begin(115200);
    randomSeed(micros());
    LittleFS.begin(true);
    loadConfig();
    WiFi.mode(WIFI_AP_STA);
    g_deviceId = normalizeMac(WiFi.macAddress());
    g_apSsid = "Sender-" + g_deviceId.substring(g_deviceId.length()
    - 8);
    setupSoftAp();
    maintainWifiSta();
    setupAiEngine();
    startWebServer();
    markReceiverStatus();
}
```

*Sender khởi động lần lượt theo đúng các phụ thuộc: file system, config, Wi-Fi, AI và web UI.*

## *A.2 Cấu trúc trạng thái sender*

```
struct SenderConfig
{
    char wifiSsid[MAX_TEXT_LEN] = "";
    char wifiPass[MAX_TEXT_LEN] = "";
    char receiverIp[32] = "";
    uint16_t receiverPort = DEFAULT_RECEIVER_PORT;
    float threshold = 0.5f;
    char deviceName[MAX_TEXT_LEN] = "Undefined";
};
```

*SenderConfig thể hiện triết lý cấu hình tối giản nhưng đủ cho vận hành thực tế.*

## *A.3 Lưu cấu hình sender xuống LittleFS*

```
void saveConfig(bool restartAfterSave)
{
    JsonDocument doc;
    doc["wifiSsid"] = g_config.wifiSsid;
    doc["wifiPass"] = g_config.wifiPass;
    doc["receiverIp"] = g_config.receiverIp;
    doc["receiverPort"] = g_config.receiverPort;
    doc["threshold"] = g_config.threshold;
    doc["deviceName"] = g_config.deviceName;
    File file = LittleFS.open(CONFIG_FILE, "w");
    serializeJson(doc, file);
}
```

*Storage trên sender đủ nhỏ gọn để phù hợp với bài toán cấu hình cục bộ.*

## *A.4 Cấu trúc SenderState ở receiver*

```
struct SenderState
{
    bool used = false;
    bool registered = false;
    bool online = false;
    bool hasTelemetry = false;
    bool pendingConfig = false;
    bool awaitingAck = false;
    char deviceId[DEVICE_ID_LEN] = "";
    char name[MAX_NAME_LEN + 1] = "";
    float temperature = NAN;
    float humidity = NAN;
    float reportedThreshold = NAN;
    float desiredThreshold = NAN;
    float aiScore = NAN;
    bool aiAnomaly = false;
};
```

*Receiver giữ trạng thái phong phú hơn sender vì nó phải điều phối cả data plane lẫn control plane.*

## *A.5 Parse gói telemetry ở receiver*

```
bool parseTelemetry(JsonObject data, SenderTelemetryPacket &packet)
```

```
{
    const String deviceId = normalizeDeviceId(String(data["device_id"] | ""));
    copyStringToBuffer(deviceId, packet.deviceId, sizeof(packet.deviceId));
    packet.temperature = data["temperature"] | NAN;
    packet.humidity = data["humidity"] | NAN;
    packet.threshold = data["threshold"] | NAN;
    packet.aiScore = data["ai_score"] | NAN;
    packet.aiAnomaly = String(data["ai_status"] | "").equalsIgnoreCase("ANOMALY");
    packet.timestampMs = data["timestamp"] | 0ULL;
    return true;
}
```

*Packet telemetry được parse thành cấu trúc trung gian trước khi hợp nhất vào SenderState.*

## A.6 Gửi gói cấu hình

```
doc["type"] = "config";
doc["device_id"] = sender.deviceId;
doc["name"] = sender.name;
doc["threshold"] = sender.desiredThreshold;
doc["receiver_ip"] = WiFi.localIP().toString();
doc["receiver_port"] = LOCAL_TCP_PORT;
doc["config_version"] = sender.configVersion;
```

*Các trường trong packet cấu hình đủ để sender biết được ngưỡng mới và địa chỉ receiver hiện hành.*

## A.7 Chủ đề MQTT gateway

```
constexpr char MQTT_GATEWAY_CONNECT_TOPIC[] = "v1/gateway/connect";
constexpr char MQTT_GATEWAY_DISCONNECT_TOPIC[] = "v1/gateway/disconnect";
constexpr char MQTT_GATEWAY_TELEMETRY_TOPIC[] = "v1/gateway/telemetry";
constexpr char MQTT_GATEWAY_ATTRIBUTES_TOPIC[] = "v1/gateway/attributes";
constexpr char MQTT_GATEWAY_RPC_TOPIC[] = "v1/gateway/rpc";
```

*Receiver gom toàn bộ giao tiếp cloud vào một nhóm topic nhất quán.*

## *A.8 Snapshot cho giao diện web*

```
doc["type"] = "snapshot";
receiver["cloudStatus"] = cloudStatusText();
receiver["wifiConnected"] = WiFi.status() == WL_CONNECTED;
receiver["wifiIp"] = WiFi.localIP().toString();
JsonArray senders = doc["senders"].to<JsonArray>();
JsonArray unknownDevices = doc["unknownDevices"].to<JsonArray>();
```

*Snapshot là lớp dữ liệu trung gian quan trọng giữa state nội bộ và UI.*

# 2 Phụ lục B: Hình ảnh thực nghiệm bổ sung

Phụ lục này trình bày các hình ảnh thực nghiệm bổ sung để minh họa rõ quá trình vận hành của hệ thống.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/hardware_layout.png`

Mục tiêu ảnh: Ảnh bố trí phần cứng thật: nhiều sender và một receiver. Nên chụp trong cùng khung hình để nhấn mạnh khả năng mở rộng của mô hình gateway.

**Hình 36:** Bố trí phần cứng của hệ thống.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/sender_alert_anomaly.png`

Mục tiêu ảnh: Ảnh thể hiện khi sender rơi vào trạng thái ANOMALY, bao gồm AI score và ngưỡng hiện tại.

**Hình 37:** Sender ở trạng thái ANOMALY.

**GHI CHÚ CHÈN ẢNH THỰC NGHIỆM**

Đường dẫn dự kiến: `assets/cloud_rpc_success.png`

Mục tiêu ảnh: Ảnh chụp kết quả RPC thành công trên CoreIoT hoặc log xác nhận từ receiver.

**Hình 38:** Xác nhận RPC thay đổi threshold thành công.